



IsaVODEs: Interactive Verification of Cyber-Physical Systems at Scale

Jonathan Julián Huerta y Munive¹ · Simon Foster² · Mario Gleirscher³ · Georg Struth⁴ · Christian Pardillo Laursen² · Thomas Hickman²

Received: 17 January 2024 / Accepted: 29 July 2024 / Published online: 19 October 2024
© The Author(s) 2024

Abstract

We formally introduce IsaVODEs (Isabelle verification with Ordinary Differential Equations), an open, compositional and extensible framework for the verification of cyber-physical systems. We extend a previous semantic approach with methods and techniques that increase its expressivity, proof automation, and scalability to the level of state-of-the-art deductive verification tools. Our contributions include a user-friendly specification language, a flexible hybrid store model, including vectors and matrices, and separation-logic-style rules for local reasoning with hybrid stores using a novel form of differentiation called framed Fréchet derivatives. The formalisation of correctness specifications with forward predicate transformers, the certification of flows as unique solutions to systems of ordinary differential equations, and invariant reasoning for such systems also contribute to the scalability and usability of our framework. In combination, these features make our framework flexible and adaptable to several verification workflows. A suite of examples and hybrid systems verification benchmarks validate our framework relative to other state-of-the-art approaches.

✉ Jonathan Julián Huerta y Munive
huertjon@cvut.cz

Simon Foster
simon.foster@york.ac.uk

Mario Gleirscher
mario.gleirscher@uni-bremen.de

Georg Struth
g.struth@sheffield.ac.uk

Christian Pardillo Laursen
christian.laursen@york.ac.uk

Thomas Hickman
Thomas.Hickman42@gmail.com

¹ Czech Institute of Informatics, Robotics and Cybernetics, Czech Technical University in Prague, Jugoslávských partyzánů 1580/3, 160 00 Prague, Czechia

² Computer Science, University of York, Deramore Lane, York YO10 5GH, UK

³ Mathematics & Computer Science, University of Bremen, Bibliothekstraße 5, 28359 Bremen, Germany

⁴ Regent Court, 211 Portobello, Sheffield S1 4DP, UK

Keywords Cyber-physical systems · Hybrid systems · Program correctness and verification · Interactive theorem proving · Predicate transformers · Lenses

1 Introduction

Cyber-physical systems (CPSs) are computerised systems whose software—the “cyber” part—interacts with their physical environment. The software is usually given by a variable-updating, potentially non-deterministic program, while the environment can be modelled by a system of ordinary differential equations (ODEs). The interaction of CPSs with humans, for example through robotic manipulators, makes them invariably safety-critical and, hence, their design verification is desirable.

Yet, CPS verification is challenging because of the complex interactions between the software, the hardware, and the physical environment. Verification is generally intractable, and thus requires abstractions. A pioneering deductive verification approach based on symbolic logics, which support the encoding of solutions and invariants of ODEs, has been implemented in the KeYmaera X tool (KYX) [28]. CPSs are modelled via hybrid programs and their behaviour is verified using differential dynamic logic ($d\mathcal{L}$) that extends standard dynamic logic by domain-specific inference rules. Numerous case studies and competitions support the applicability of this approach [38, 43, 47, 77].

Alternatively, general-purpose interactive theorem provers (ITPs), like Coq, Lean or Isabelle, have more recently been used for building CPS verification components [20, 49, 50, 78]. Isabelle, in particular, provides extensive libraries for analysis and ODEs for this purpose [32, 36].

ITP-based approaches offer compelling advantages. Their universality, based on typed higher-order logics, makes them expressive for formalising the mathematical components needed. Their openness allows large user communities to contribute domain-specific libraries and verification tools in incremental and compositional ways. Their trustworthiness allows such components to be built as conservative extensions over small kernels that have passed the test of time. Prime examples for these features are Isabelle’s Archive of Formal Proofs and Lean’s Mathematical Library.¹

We have previously introduced a semantic framework as a proof of concept that $d\mathcal{L}$ -inspired verification of cyber-physical systems can be achieved in general-purpose provers [20, 49, 50] with little formalisation effort relative to extant mathematical and verification components. It simplifies and streamlines the $d\mathcal{L}$ semantics and proof infrastructure by simply adding a hybrid store model to algebraic or categorical predicate transformer semantics and their specialisations to Hoare logics and refinement calculi [7, 30]. The resulting prototypical verification tool for hybrid systems integrates the Isabelle components for analysis and ODEs mentioned [32, 36], using a shallow embedding, which is agnostic and easily adaptable to syntactic user preferences.

Yet, it remains to transform this prototype into a scalable verification tool for cyber-physical systems that combines user-friendly specification features with increased proof automation for reasoning with the dynamical systems involved.

Contributions In this article, we evolve our previous prototype [20, 50] into an Isabelle-based verification framework called IsaVOEs (Isabelle Verification with Ordinary Differential Equations). IsaVOEs provides a textual language for modelling CPSs, constructed as a

¹ Isabelle’s Archive of Formal Proofs at <https://www.isa-afp.org/statistics/>. Lean’s Mathematical Library at https://leanprover-community.github.io/mathlib_stats.html.



shallow embedding in Isabelle. This language is based on the previously introduced state- and predicate-transformer semantics for hybrid programs [50] and on a flexible lens-based hybrid store model [21, 25]. The semantics supports reasoning in the style of dynamic logic, Hoare logic, or refinement calculi, with modalities for specifying both safety and reachability properties. The store model allows software models to benefit from the full generality of Isabelle's type system, including continuous structures like vectors and matrices, and discrete structures like algebraic data types. Our lens-based approach also facilitates local reasoning à la separation logic, where a verification task can be decomposed according to the dynamic resources a program module uses. This integration makes our language scale to systems that include both realistic non-linear dynamics and complex control structures. We harness Isabelle's syntax translation mechanisms to provide a user-friendly front-end for the language, including declarative context, program notation, and operators for arithmetic, vectors, and matrices. Our technical solution is extensible and inspired by languages like Modelica, Mathematica or MATLAB.

Verification of cyber-physical systems in IsaVODEs is supported by a substantial library of domain-specific deduction rules that are correct by construction within Isabelle. These can be used in addition to the standard rules for predicate transformers or Hoare logic. For reasoning about ODEs, we support two workflows: the first supplies solutions via flows for ODEs, while the second is based on reasoning with invariant sets of ODEs and thus avoids the need for solutions [71]. To enhance the first workflow, we integrate two Computer Algebra Systems (CASs), Mathematica and SageMath, into Isabelle. These may supply flows for Lipschitz-continuous ODEs in a way analogous to Isabelle's Sledgehammer. For the second workflow, we provide a $d\mathcal{L}$ -style differential ghost rule and Darboux rules that enhance IsaVODEs' invariant reasoning capabilities for continuous dynamics where explicit solutions may not exist.

Several theoretical contributions transcend the engineering aspects of our framework. We provide novel laws for local reasoning with hybrid stores, which allow us to infer the frame of a program (i.e. the mutable variables), and use a frame rule to prove invariants with respect to variables outside the frame. This is complemented by a novel form of differentiation called “framed Fréchet derivatives”, which allows us to differentiate with respect to a strict subset of the store variables, particularly ignoring any discrete variables. When variables are outside the frame of a system of ODEs, those variables are unchanged during a continuous evolution. Local reasoning further enhances the scalability of our tool by allowing a verification task to be decomposed into modules with their own local pre- and postconditions.

We also supply several proof methods to support automated proof and verification condition generation (VCG), including various Hoare-logic and weakest precondition laws, the application of differential induction, and certification of ODE solutions. Care is taken to ensure that the resulting proof obligations are presented in a way that preserves abstraction, and elides irrelevant details of the Isabelle mechanisation, to support the user. Harnessing Isabelle's Eisbach tool [44], we also employ various high-level search-based proof methods, which exhaustively apply the proof rules to compute a complete set of VCs. The VCs can finally be tackled in the usual way using tools like auto and Sledgehammer.

To evidence the scalability and automation of our framework, we have evaluated it on a large set of hybrid verification benchmarks [45], and many non-trivial examples. Our enhancements lead at least to the same performance in essential verification tasks as that of other state-of-the-art provers [47]. Initial case studies suggest a simplification of user

interaction. Our new components can be found online,² across the text. Our contributions are highlighted through examples, while additional contributions are noted throughout the text.

This article is a substantial extension of a previous conference paper at (FM2021) [21]. Additional contributions include generalisations of the laws for frames (Sect. 5.1), differential ghosts, and Darboux rules (Sect. 5.4); proof methods for certification of continuity, Lipschitz continuity, and uniqueness of solutions (Sect. 6.2); extensions to our previous proof methods (Sect. 6); the CAS integrations (Sect. 6.5); a more substantial evaluation (Sect. 7); and several additional examples (Sects. 2, 8). As a result of these enhancements, IsaVODEs has become much more powerful and user friendly.

Overview We begin our paper by demonstrating the usage of our framework with a small case study (Sect. 2). We then expound the foundations of our formalisation: dynamical systems (Sect. 3.1), the state-transformer hybrid program model (Sect. 3.2), the basics of the store model (Sect. 3.3), the specification of continuous evolutions (Sect. 3.4), and finally VCG through our framework’s algebraic foundations (Sect. 3.5) via predicate transformers. This includes safety, reachability and termination (Sect. 3.5.3). Next, in Sect. 4, we introduce our framework’s hybrid modelling language. We supply commands for generating hybrid stores, with variables, constants, and foundational properties (Sect. 4.1), and user-friendly notation for expressions (Sect. 4.3), matrices, and vectors (Sect. 4.4). The notation is automatically processed via rewriting rules supplied to Isabelle’s simplifier, hiding implementation details.

In Sect. 5, we present laws for local reasoning about frames (Sect. 5.1), the seamless integration of framed ODEs into our hybrid programs (Sect. 5.2), and automatic certification of ODE invariants through framed Fréchet derivatives (Sect. 5.3). These also serve to supply new $d\mathcal{L}$ -style ghost and Darboux rules that enhance IsaVODEs’ invariant reasoning for the second workflow (Sect. 5.4). We contribute increased IsaVODEs proof-capabilities for the verification process. We formalise theorems necessary for increased automation on the first workflow, like the fact that differentiable functions are Lipschitz-continuous (Sect. 6.2) or the first and second derivative test laws (Sect. 6.7). We provide methods to automate certification of differentiation (Sect. 6.1), uniqueness of solutions (Sect. 6.3), invariants for ODEs (Sect. 6.6), and VCG (Sect. 6.4). Additional automation is provided through the integration of Mathematica and SageMath (Sect. 6.5). We then bring all of the results together for the evaluation (Sect. 7) using benchmarks and examples (Sect. 8). In the remaining sections we review related work (Sect. 9), provide concluding statements, and discuss future research avenues (Sect. 10).

2 Motivating Case Study: Flight Dynamics

In this section, we motivate and demonstrate our framework’s usage with a worked example: an aircraft collision avoidance scenario that was first presented by Mitsch et. al. [46]. It describes an aircraft trying to avoid a collision with a nearby intruding aircraft travelling at the same altitude. We can model this intruding aircraft by considering its coordinates x and y , and angle ϑ , in the reference frame of our own ship. The intruding plane has fixed velocity $v_i > 0$, and our plane has fixed velocity $v_o > 0$ and variable angular velocity ω . This is illustrated in Fig. 1a and b.

² See <https://github.com/isabelle-utp/Hybrid-Verification> also by clicking our icons  .

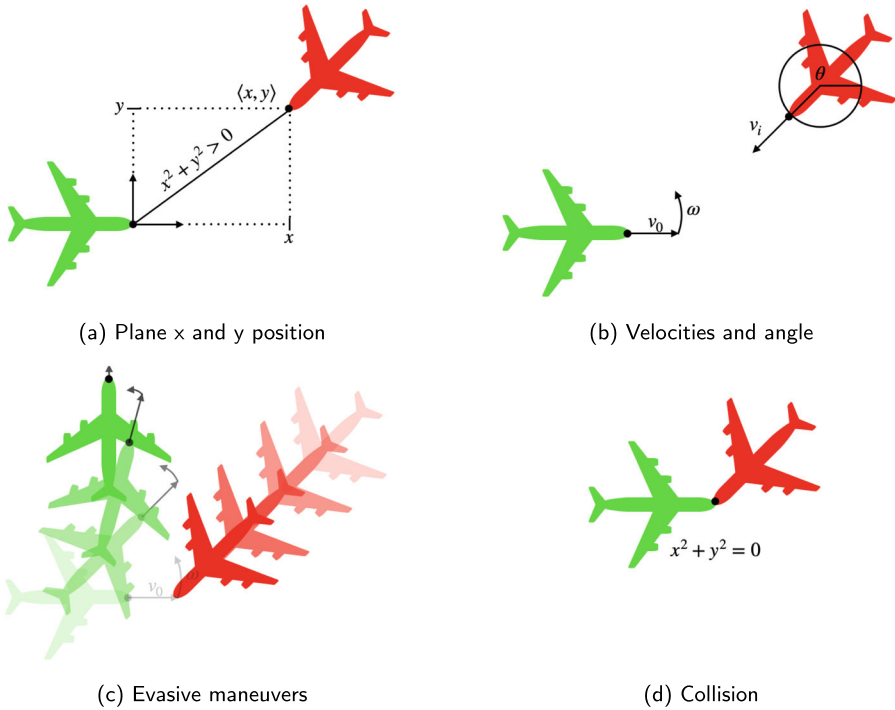


Fig. 1 Diagrams illustrating the flight dynamics example

With our tool, we can model this using a **dataspace** command (see Sect. 4.1), as shown below:

```
dataspace planar_flight =
  constants
    v_o :: real (* own_velocity *)
    v_i :: real (* intruder velocity *)
  assumes
    v_o_pos: "v_o > 0" and
    v_i_pos: "v_i > 0"
  variables (* Coordinates in reference frame of own ship *)
    x :: real (* Intruder x *)
    y :: real (* Intruder y *)
    theta :: real (* Intruder angle *)
    omega :: real (* Angular velocity *)
```

This command allows us to define our constants, any assumptions about the verification problem, and the system's variables. In this case, we postulate two constants v_o and v_i , both of type **real**, for the velocity of the aircraft and intruder, respectively. Here, **real** is the type of precise mathematical real numbers, as opposed to floating points or rationals.

As per the problem statement, we assume that both of these constants are strictly positive. This is specified by the assumptions called v_o_pos and v_i_pos , respectively, which can be used as hypotheses in proofs. We supply the variables of this system, which give the relative position of the intruder (x and y), its orientation θ , and its angular velocity ω .

We next specify the dynamics that model the physical system by defining a constant called `plant`. The system of ODEs is non-linear and uses trigonometric functions:

definition "plant $\equiv \{x' = v_i * \cos \theta - v_o + \omega * y,$
 $y' = v_i * \sin \theta - \omega * x,$
 $\theta' = -\omega\}$ "

The ODEs can be specified in a user-friendly manner, as physicists and engineers would informally state them, in terms of the constants and variables of the system x , y , and φ .

Next, we define a simple controller to avoid collisions, as explained below:

abbreviation "I $\equiv (v_i * \sin \theta * x - (v_i * \cos \theta - v_o) * y$
 $> v_o + v_i)^e$ "

abbreviation "J $\equiv (v_i * \omega * \sin \theta * x - v_i * \omega * \cos \theta * y$
 $+ v_o * v_i * \cos \theta$
 $> v_o * v_i + v_i * \omega)^e$ "

definition "ctrl $\equiv (\omega ::= 0; \zeta I?) \sqcap (\omega ::= 1; \zeta J?)$ "

This is based on two invariants, I and J , for two different scenarios. These are properties that remain true after the unfolding of each scenario. The invariant I holds when going straight is safe, while J holds when an evasion manoeuvre is allowed. Our controller selects whether to set our aircraft's angular velocity to 0 or 1 depending on which invariant holds (I and J respectively). The evasive manoeuvres that occur when the angular velocity is set to 1 are illustrated in Fig. 1c.

Finally, our model follows the usual structure of an iteration (*) of control choices (ctrl) followed by a nondeterministic evolution of the system dynamics (plant):

definition "flight $\equiv (\text{ctrl}; \text{plant})^*$ "

The behaviour of the system is characterised by all states reachable by executing the controller followed by the plant a finite number of times. Our contributions in Sects. 4 and 5 support the smooth declaration of these definitions.

Next, we show how we can formally verify the collision avoidance of this system. We do this by specifying a Hoare triple within an Isabelle lemma: 

```
lemma flight_safe: "{x2 + y2 > 0} flight {x2 + y2 > 0}"
proof -
  have ctrl_post: "{x2 + y2 > 0} ctrl {(\omega = 0 \wedge @I) \vee (\omega = 1 \wedge @J)}"
    unfolding ctrl_def by wlp_full

  have plant_safe_I: "{\omega = 0 \wedge @I} plant {x2 + y2 > 0}"
    unfolding plant_def apply (dInv "$\omega = 0 \wedge @I$", dWeaken)
    using v_o_pos v_i_pos sum_squares_gt_zero_iff by fastforce

  have plant_safe_J: "{\omega = 1 \wedge @J} plant {x2 + y2 > 0}"
    unfolding plant_def apply (dInv "$\omega = 1 \wedge @J$", dWeaken)
    by (smt (z3) cos_le_one mult_if_delta mult_le_cancel_iff2
      mult_left_le sum_squares_gt_zero_iff v_i_pos v_o_pos)

  show ?thesis
    unfolding flight_def
    apply (intro hoare_kstar_inv hoare_kcomp[OF ctrl_post])
    by (rule hoare_disj_split[OF plant_safe_I plant_safe_J])
qed
```

We formulate collision avoidance using $x^2 + y^2 > 0$ as the invariant in the Hoare triple. A collision occurs when $x^2 + y^2 = 0$, as illustrated in figure 1d. We use Isabelle’s Isar scripting language to break down the verification into several intermediate properties via the Isar command “**have**”, which takes a label followed by a property specification.

We first calculate the postconditions that arise from running the controller (`ctrl`) by using our tactic `wlp_full` (see Sect. 6). This gives us two possible execution branches—one where $I \wedge \omega = 0$ holds, and one where $J \wedge \omega = 1$ holds. We give this first property the name `ctrl_post`.

The postconditions provide two possible initial states for the plant, which we consider using the properties `plant_safe_I` and `plant_safe_J`. We show that both preconditions guarantee the problem’s postcondition $x^2 + y^2 > 0$ by applying differential induction, a technique that proves an invariant of a system of ODEs without computing a solution (see Sect. 5.3). In each case, we use our Eisbach-designed method `dInv`, which takes as a parameter the invariant we wish to prove. The invariant is simply the precondition of each Hoare triple. However, we then need to prove that this invariant implies the postcondition, which is done using a further method called `dWeaken`. The remaining proof obligations can be solved using the **Sledgehammer** tool, which calls external automated theorem provers to find a solution and reconstructs it using the names of previously proven theorems in Isabelle’s libraries. In particular, the property `plant_safe_J` is discharged with a call to **Sledgehammer**, which uses trigonometric identities formalised in **HOL-Analysis**. We display the proof state resulting from applying differential induction below:

```
`$ω = 1 ∧
  v_o * v_i + v_i * $ω
  < v_i * $ω * sin ($θ) * $x - v_i * $ω * cos ($θ) * $y +
    v_o * v_i * cos ($θ) →
  0 < ($x)^2 + ($y)^2`
```

In the case that **Sledgehammer** is unable to find a proof, this proof obligation is quite readable, and so a manual proof or refutation could be given (see Sect. 8).

Finally, we put everything together to show that the whole system is safe, using the Isar command **show**, which is used to conclude proofs and restate the overall goal of the lemma (`?thesis`). The proof uses a couple of high-level Hoare logic laws, and the properties that were proven to complete the proof. This final step can be completely automated using Isabelle’s classical reasoner, but we leave the details for the purpose of demonstration.

It is also possible to provide a proof of this verification problem using the aforementioned first workflow. Namely, users can provide the solutions to the system of ODEs `plant` as we do for other problems throughout the article (see Sects. 5.2, 8). This completes our overview of our tool and its capabilities. In the remainder of the paper, we expound the technical foundations of the tool, and our key results.

3 Semantics for Hybrid Systems Verification

We start the section by recapitulating basic concepts from the theory of dynamical systems. We use these notions to describe our approach [21, 50] to hybrid systems verification in general-purpose proof assistants via predicate transformers. Specifically, we present hybrid programs and their state transformer semantics. We then introduce our store model and provide intuitions for deriving state transformers for program assignments and ODEs relative

to this model. We extend our approach to a predicate transformer semantics to derive laws for verification condition generation (VCG) [7, 50]. That is, we present the forward box predicate transformer and use it to derive the rules of Hoare logic. Finally, we introduce the forward diamond predicate transformer which models reachability and progress properties of hybrid systems. Our formalisation of these concepts as a shallow embedding in Isabelle maximises the availability of the prover's proof automation in VCG.

3.1 Dynamical Systems

In this section, we consider two ways to specify continuous dynamical systems [71]: explicitly via flows and implicitly via systems of ordinary differential equations (ODEs). *Flows* are functions $\varphi : T \rightarrow \mathcal{C} \rightarrow \mathcal{C}$, where T is a non-discrete submonoid of the real numbers $\mathbb{R}$ representing time. Similarly, $\mathcal{C}$ is a set with some topological structure like a vector space or a metric space. We emphasise this intuition using an overhead arrow for elements $\vec{c} \in \mathcal{C}$ and refer to $\mathcal{C}$ as a *continuous* state space. By definition, flows are continuously differentiable (C^1) functions and monoid actions: they satisfy the laws

$$\varphi(t_1 + t_2) = \varphi t_1 \circ \varphi t_2 \quad \text{and} \quad \varphi 0 = id_{\mathcal{C}}.$$

Given a state $\vec{c} \in \mathcal{C}$, the *trajectory* $\varphi_{\vec{c}} : T \rightarrow \mathcal{C}$, such that $\varphi_{\vec{c}} t = \varphi t \vec{c}$, is a curve modelling the continuous dynamical system's evolution in time and passing through $\vec{c}$. The *orbit map* $\gamma^{\varphi} : \mathcal{C} \rightarrow \mathcal{P} \mathcal{C}$, such that $\gamma^{\varphi} \vec{c} = \mathcal{P} \varphi_{\vec{c}} T$, gives the graph of this curve for $\vec{c}$, where $\mathcal{P}$ denotes both the powerset operation and the direct image operator.

Systems of ODEs are related to flows through their solutions. Formally, systems of ODEs are specified by *vector fields* $f : T \rightarrow \mathcal{C} \rightarrow \mathcal{C}$, functions assigning vectors to points in space-time. A *solution* X to the system of ODEs specified by f is then a C^1 -function

$$X : T \rightarrow \mathcal{C} \quad \text{such that} \quad X' t = f t (X t),$$

for all t in some interval $U \subseteq T$. This solution also solves the associated *initial value problem* (IVP) given by $(t_0, \vec{c})$ (denoted by $X \in \text{ivp-sols } U \text{ } f \text{ } t_0 \vec{c}$) if it satisfies

$$X t_0 = \vec{c} \quad \text{with} \quad t_0 \in U.$$

The existence of solutions to IVPs is guaranteed for continuous vector fields by the Peano theorem, albeit on an interval $U \vec{c}$ depending on the initial condition $(t_0, \vec{c})$. Similarly, the Picard–Lindelöf theorem states that all solutions to the IVP $X' t = f t (X t)$ with $X t_0 = \vec{c}$ coincide in some interval $U \vec{c} \subseteq T$ (around t_0) if f is Lipschitz continuous on T . In other words, it states that there is a unique solution to the IVP on $U \vec{c}$. Thanks to this, when f is Lipschitz continuous on T , $t_0 = 0$, and $U \vec{c} = T$ for all $\vec{c} \in \mathcal{C}$, the solutions to the associated IVPs are exactly a flow's trajectories $\varphi_{\vec{c}}$, that is

$$\varphi'_{\vec{c}} t = f t (\varphi_{\vec{c}} t) \quad \text{and} \quad \varphi_{\vec{c}} 0 = \vec{c}.$$

Therefore, the flow φ is the function mapping to each $\vec{c} \in \mathcal{C}$ the unique $\varphi_{\vec{c}}$ such that $\varphi_{\vec{c}} \in \text{ivp-sols } T \text{ } f \text{ } 0 \vec{c}$.

Example 1 We illustrate the above properties about flows and ODEs with the equation

$$y' t = a \cdot y t + b,$$

where $a \neq 0$ and $t \in \mathbb{R}$. This is an important ODE modelling for instance idealised bacterial growth [69], radioactive decay [74] or concentration of glucose in the blood (without the

intervention of insulin) [2]. First, given that differentiability implies Lipschitz continuity [71], we can verify that this equation has unique solutions (by the Picard–Lindelöf theorem) simply by noticing that $f\ t = a \cdot y\ t + b$ is differentiable with derivative $f'\ t = a \cdot y'\ t$. The solution to the associated IVP with initial condition $y\ 0 = c$ is

$$\varphi_c\ t = \frac{b}{a} \cdot e^{a \cdot t} + c \cdot e^{a \cdot t} - \frac{b}{a}.$$

Indeed,

$$\begin{aligned} \varphi'_c\ t &= \left(\frac{b}{a} \cdot e^{a \cdot t} + c \cdot e^{a \cdot t} - \frac{b}{a} \right)' \\ &= \frac{b}{a} \cdot a \cdot e^{a \cdot t} + c \cdot a \cdot e^{a \cdot t} \\ &= a \cdot \left(\frac{b}{a} \cdot e^{a \cdot t} + c \cdot e^{a \cdot t} - \frac{b}{a} \right) + b \\ &= a \cdot \varphi_c\ t + b. \end{aligned}$$

It is also easy to check that $\varphi_c\ 0 = c$. Hence, the mapping $\varphi : \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$, such that $\varphi\ t\ c = \varphi_c\ t$, is the unique flow associated to the ODE $y'\ t = a \cdot y\ t + b$. Indeed, for a fixed $\tau \in \mathbb{R}$, the function $g : \mathbb{R} \rightarrow \mathbb{R}$ such that $g\ t = \varphi\ (t + \tau)\ c = \varphi_c\ (t + \tau)$ satisfies

$$g\ 0 = \varphi_c\ \tau \quad \text{and also} \quad g'\ t = \varphi'_c\ (t + \tau) = a \cdot \varphi_c\ (t + \tau) + b = a \cdot (g\ t) + b.$$

However, by uniqueness, the only function satisfying these two equations is $\varphi_{\varphi_c\ \tau}$. Hence $g\ t = \varphi_{\varphi_c\ \tau}\ t$ and the monoid law holds: $\varphi\ (t + \tau)\ c = \varphi\ t\ (\varphi\ \tau\ c)$. Thus, the function φ mapping points c to IVP-solutions φ_c of the ODE $y'\ t = a \cdot y\ t + b$ satisfies the monoid action laws, and is therefore, a flow. $\square$

3.2 State Transformer Semantics for Hybrid Programs

Having introduced some basic concepts from the theory of dynamical systems, we present our hybrid systems model. We represent these via hybrid programs which are traditionally defined syntactically [31, 61]. Yet, our approach is purely semantic and we merely provide the recursive definition below as a guide to what our semantics should model:

$$\alpha ::= x := e \mid x' = f \ \& \ G \mid \iota P? \mid \alpha ; \alpha \mid \alpha \sqcap \alpha \mid \alpha^*.$$

Typically, x denotes variables; e and f are terms, and G and P are assertions. In dynamic logic [31], the statement $x := e$ represents an *assignment* of variable x to expression e , $\iota P?$ models testing whether P holds, while $\alpha ; \beta$, $\alpha \sqcap \beta$ and α^* are the *sequential composition*, *non-deterministic choice*, and *finite iteration* of programs α and β . Well-known while-programs emerge via the equations

$$\begin{aligned} \text{if } P \text{ then } \alpha \text{ else } \beta &\equiv (\iota P? ; \alpha) \sqcap (\iota \neg P? ; \beta) \text{ and} \\ \text{while } P \text{ do } \alpha &\equiv (\iota P? ; \alpha)^* ; \iota \neg P? \end{aligned}$$

Beyond these, differential dynamic logic (dL) [61] uses *evolution commands* $x' = f \ \& \ G$ that represent systems of ODEs with *boundary conditions* or *guards* G . A guard G delimits the solutions' range to the region described by G .

We use *nondeterministic state transformers* $\alpha : \mathcal{S} \rightarrow \mathcal{P}\mathcal{S}$ as our semantic representation for hybrid programs. Thus, our “hybrid programs” are really arrows in the Kleisli category of

the powerset monad. Observe that a subset of these arrows also model “assertions”, namely the *subidentities* of the monadic unit η_S , such that $\eta_S s = \{s\}$ for all $s \in S$. That is, the functions mapping each state s either to $\{s\}$ or to $\emptyset$ model assertions where $P s = \{s\}$ represents that P holds for s , and $P s = \emptyset$ that P does not hold for s . Henceforth, we abuse notation and identify predicates $P : S \rightarrow \mathbb{B}$ (or $P \in \mathbb{B}^S$), sets ($P \in \mathcal{P} S$), and subidentities of η_S , where $\mathbb{B}$ denotes the Booleans. We also treat predicates as logic formulae by writing, for instance, $P \wedge Q$ and $P \vee Q$ instead of $\lambda s. P s \wedge Q s$ and $\lambda s. P s \vee Q s$. Thus, we denote the constantly true and constantly false predicates by $\top$ and $\perp$ respectively. They coincide with the **skip** and **abort** programs such that **skip** $= \eta_S$ and **abort** $s = \emptyset$ for all $s \in S$. In this state transformer semantics, sequential compositions correspond to (forward) *Kleisli compositions*, nondeterministic choices are point-wise unions, and finite iterations are a limit of sequential compositions. That is,

$$\begin{aligned} (\alpha ; \beta) s &= \bigcup \{ \beta s' \mid s' \in \alpha s \}, \\ (\alpha \sqcap \beta) s &= \alpha s \cup \beta s, \text{ and} \\ \alpha^* s &= \bigcup_{i \in \mathbb{N}} \alpha^i s, \end{aligned}$$

with $\alpha^{i+1} = \alpha ; \alpha^i$ and $\alpha^0 = \text{skip}$.

3.3 Store and Expressions Model

To introduce our state transformer semantics of assignments and ODEs, we first describe our store model. We use lenses [8, 24, 52] to algebraically characterise the hybrid store. Through an axiomatic approach to accessing and mutating functions, lenses allow us to locally manipulate program stores [18] and algebraically reason about program variables [23, 25]. Formally, a *lens* x with *source* S and *view* $\mathcal{V}$, denoted $x :: \mathcal{V} \Rightarrow S$, is a pair of functions (get_x, put_x) with $get_x : S \rightarrow \mathcal{V}$ and $put_x : \mathcal{V} \rightarrow S \rightarrow S$ such that

$$get_x (put_x v s) = v, \quad put_x v \circ put_x v' = put_x v, \quad \text{and} \quad put_x (get_x s) s = s, \quad (\text{lens-laws})$$

for all $v, v' \in \mathcal{V}$ and $s \in S$. Usually, a lens x represents a variable, S is the system’s state space and $\mathcal{V}$ is the value domain for x . Under this interpretation, get_x returns the value of variable x while put_x updates it. Yet, we sometimes interpret $\mathcal{V}$ as a subregion of S , making get_x a projection or restriction and put_x an overwriting or substituting function. There are other models using diverse variable lenses, such as arrays, maps and local variables [18, 25].

We model expressions and assertions used in program syntax as functions $e : S \rightarrow \mathcal{V}$, which semantically are queries over the store S returning a value of type $\mathcal{V}$. We can use the get function to perform such queries by “looking up the value of variables”. For instance, if $x, y :: \mathbb{R} \Rightarrow S$ model independent program variables, and $c \in \mathbb{R}$ is a constant, then the function $\lambda s. (get_x s)^2 + (get_y s)^2 \leq c^2$ represents the “expression” $x^2 + y^2 \leq c^2$. Then, function evaluation corresponds to computing the value of the expression at state $s \in S$.

With this representation, the state transformer $\lambda s. \{put_x (e s) s\}$ models a program assignment, denoted by $x := e$. More generally, we turn deterministic functions $\sigma : S \rightarrow S$ into state transformers via function composition with the Kleisli unit, which we denote by $\langle \sigma \rangle = \eta_S \circ \sigma$. Then, representing expressions e as functions $e : S \rightarrow \mathcal{V}$, our model for variable assignments becomes

$$(x := e) = \langle \lambda s. put_x (e s) s \rangle.$$

3.4 Model for Evolution Commands

To derive in our framework a state transformer semantics $\mathcal{S} \rightarrow \mathcal{P}\mathcal{S}$ for evolution commands $x' = f \ \& \ G$, observe that a flow's orbit map $\gamma^\varphi : \mathcal{C} \rightarrow \mathcal{P}\mathcal{C}$ with $\gamma^\varphi \vec{c} = \{\varphi t \vec{c} \mid t \in T\}$ is already a state transformer on $\mathcal{C}$. It sends each $\vec{c}$ in the continuous state space $\mathcal{C}$ to the reachable states of the trajectory $\varphi \vec{c}$. Based on the relationship between flows and the solutions to IVPs from Sect. 3.1, we can generalise this state transformer to a set of all points $X t$ of the solutions X of the system of ODEs represented by f , i.e. $X' t = f t (X t)$, with initial condition $X t_0 = \vec{c}$ over an interval $U \vec{c}$ around t_0 ($t_0 \in U \vec{c}$). Moreover, in line with $d\mathcal{L}$, the solutions should remain within the guard or boundary condition G : $\forall \tau \in U \vec{c}. \tau \leq t \Rightarrow G (X \tau)$. Thus, to specify dynamical systems via ODEs f instead of flows φ , we define the *generalised guarded orbits* map [50]

$$\gamma^f G U t_0 \vec{c} = \{X t \mid t \in U \vec{c} \wedge X \in \text{ivp-sols } U f t_0 \vec{c} \wedge \mathcal{P} X (t \downarrow_U \vec{c}) \subseteq G\}, \quad (\text{orbit-map})$$

that also introduces guards G , initial conditions t_0 , and intervals $U \vec{c}$. The set $t \downarrow_T$ is a downward closure $t \downarrow_T = \{\tau \in T \mid \tau \leq t\}$ which in applications becomes the interval $[0, t] = \{\tau \mid 0 \leq \tau \leq t\}$ because we usually fix $U \vec{c} = \mathbb{R}_{\geq 0} = \{\tau \mid \tau \geq 0\}$ for all $\vec{c} \in \mathcal{C}$. This is why we also abuse notation and write constant interval functions $\lambda \vec{c}. T$ simply as T . Notice that when the flow φ for f exists, $\gamma^f \top T 0 \vec{c} = \gamma^\varphi \vec{c}$.

Lenses support algebraic reasoning about variable frames: the set of variables that a hybrid program can modify. In particular, they allow us to split the state space into continuous and discrete parts. We explain a lens-based lifting of guarded orbits $\gamma^f G U t_0$ from the continuous space $\mathcal{C} \rightarrow \mathcal{P}\mathcal{C}$ to the full state space $\mathcal{S}$ in Sect. 5.2. This produces our state transformer for evolution commands $(x' = f \ \& \ G)_{\mathcal{U}}^0 : \mathcal{S} \rightarrow \mathcal{P}\mathcal{S}$. Intuitively, it maps each state $s \in \mathcal{S}$ to all X -reachable states within the region G , where X solves the system of ODEs specified by f , and leaves intact the non-continuous part of $\mathcal{S}$. Having the same type as the above-described state transformers enables us to seamlessly use the same operations on $(x' = f \ \& \ G)_{\mathcal{U}}^0$. This also enables us to do modular verification condition generation (VCG) of hybrid systems as described in Sect. 3.5.1.

Example 2 In this example, we use a hybrid program `blood_sugar` to model an idealised machine controlling the concentration of glucose in a patient's body. Hybrid programs are often split into discrete control `ctrl` and continuous dynamics `dyn`. Their composition is then wrapped in a finite iteration:

$$\text{blood_sugar} = (\text{ctrl} ; \text{dyn})^*.$$

For the control, we use a conditional statement reacting to the patient's blood-glucose. Concretely, the program

$$\text{ctrl} = \text{if } g \leq g_m \text{ then } g := g_M \text{ else skip}$$

states that if the value of the patient's blood-glucose concentration g is below a certain warning threshold $g_m \geq 0$, the maximum healthy dose of insulin, represented as an immediate spike to the patient's glucose $g := g_M$, is injected into the patient's body. Otherwise, the patient is fine and the machine does nothing. The continuous variable g follows the dynamics in Example 1: $y' t = a \cdot y t + b$. We assume $a = -1$ and $b = 0$ so that the concentration of glucose decreases over time. This results in the evolution command

$$\text{dyn} = (g' = -g \ \& \ \top)_{\mathbb{R}_{\geq 0}}^0,$$

which we abbreviate as $\text{dyn} = (g' = -g)$. Formally, the assignment $g := g_M$ is the state transformer $\lambda s. \{put_g g_M s\}$, the test $g \leq g_m$ is the predicate $\lambda s. get_g s \leq g_m$, and the evolution command $g' = -g$ is the orbit map γ^φ lifted to the whole space $\mathcal{S}$, where $\varphi t c = c \cdot e^{-t}$ for all $t \in \mathbb{R}$. $\square$

3.5 Predicate Transformer Semantics

Finally, we extend our state transformer semantics to a *predicate transformer* $(\mathbb{B}^{\mathcal{S}} \rightarrow \mathbb{B}^{\mathcal{S}})$ semantics for verification condition generation (VCG). Concretely, we define two predicate transformers and use the definition of the first one for deriving partial correctness laws, including the rules of Hoare logic, and the definition of the second one for deriving reachability laws. We also exemplify the application of these to VCG.

3.5.1 Forward Boxes

We define dynamic logic's *forward box* or *weakest liberal precondition* (wlp) $[-]$ – operator as the predicate transformer $[\alpha] : \mathbb{B}^{\mathcal{S}} \rightarrow \mathbb{B}^{\mathcal{S}}$ such that

$$[\alpha] Q s \Leftrightarrow (\forall s'. s' \in \alpha s \Rightarrow Q s'),$$

for $\alpha : \mathcal{S} \rightarrow \mathcal{P} \mathcal{S}$ and $Q : \mathcal{S} \rightarrow \mathbb{B}$. It is true for those initial system's states that lead to a state satisfying Q after executing α , if α terminates. Well-known-laws are derivable and are simple consequences from this and our previous definitions [50]. These laws allow us to automate and do VCG much more efficiently than by doing Hoare Logic since all of them, except for the loop rule, are equational simplifications from left to right:

(wlp-skip)	$[\text{skip}] Q$	$=$	Q	
(wlp-abort)	$[\text{abort}] Q$	$=$	$\top$	
(wlp-test)	$[i P?] Q$	$=$	$P \Rightarrow Q$	
(wlp-assign)	$[x := e] Q$	$=$	$Q[e/x]$	
(wlp-seq)	$[\alpha ; \beta] Q$	$=$	$[\alpha][\beta] Q$	
(wlp-choice)	$[\alpha \sqcap \beta] Q$	$=$	$[\alpha] Q \wedge [\beta] Q$	
(wlp-loop)	$[\text{loop } \alpha] Q$	$=$	$\forall n. [\alpha^n] Q$	
(wlp-cond)	$[\text{if } T \text{ then } \alpha \text{ else } \beta] Q$	$=$	$(T \Rightarrow [\alpha] Q) \wedge (\neg T \Rightarrow [\beta] Q)$	

Here, n is a natural number (i.e. $n \in \mathbb{N}$), $\text{loop } \alpha$ is simply α^* , and $Q[e/x]$ is our abbreviation for the function $\lambda s. Q(put_x(e s) s)$ that represents the value of Q after variable x has been updated by the value of evaluating e on $s \in \mathcal{S}$. We write this semantic operation as a substitution to resemble Hoare logic (see Sect. 4.3). Similarly, the *wlp-law* for evolution commands informally corresponds to

$$(\text{wlp-evol}) \quad [(x' = f \& G)_U^{t_0}] Q s \Leftrightarrow \begin{array}{l} \forall X \in \text{ivp-sols } U f t_0 s. \forall t \in U s. \\ (\forall \tau \in t \downarrow_{U s}. G(X \tau)) \Rightarrow Q(X t). \end{array} \quad \text{🤖}$$

That is, a postcondition Q holds after an evolution command starting at (t_0, s) , if and only if, the postcondition holds $Q(X t)$ for all solutions to the IVP $X' t = f t(X t)$, $X t_0 = s$, for all times in the interval $t \in U s$ whose previous times respect G . Notice that, if there is a flow $\varphi : T \rightarrow \mathcal{C} \rightarrow \mathcal{C}$ for f and $U = T = \mathbb{R}_{\geq 0}$, this simplifies to

$$(\text{wlp-flow}) \quad [x' = f \& G] Q s \Leftrightarrow (\forall t \geq 0. (\forall \tau \in [0, t]. G(\varphi_s \tau)) \Rightarrow Q(\varphi_s t)). \quad \text{🤖}$$

See Sect. 5 for the formal version of these laws.

3.5.2 Hoare Triples

It is well-known that Hoare logic can be derived from the forward box operator of dynamic logic [31]. Thus, we can also write partial correctness specifications as Hoare-triples with our forward box operators via

$$\{P\} \alpha \{Q\} \Leftrightarrow (P \Rightarrow wlp \alpha Q).$$

From our wlp -laws and definitions, the Hoare logic rules below hold:

(h-skip)		$\{P\} \text{ skip } \{P\}$	
(h-abort)		$\{P\} \text{ abort } \{Q\}$	
(h-test)		$\{P\} \text{ ? } Q \text{ ? } \{P \wedge Q\}$	
(h-assign)		$\{Q[e/x]\} x := e \{Q\}$	
(h-seq)	$\{P\} \alpha \{R\} \wedge \{R\} \beta \{Q\} \Rightarrow$	$\{P\} \alpha ; \beta \{Q\}$	
(h-choice)	$\{P\} \alpha \{Q\} \wedge \{P\} \beta \{Q\} \Rightarrow$	$\{P\} \alpha \sqcap \beta \{Q\}$	
(h-loop)	$\{I\} \alpha \{I\} \Rightarrow$	$\{I\} \text{ loop } \alpha \{I\}$	
(h-cons)	$(P_1 \Rightarrow P_2) \wedge (Q_2 \Rightarrow Q_1) \wedge \{P_2\} \alpha \{Q_2\} \Rightarrow$	$\{P_1\} \alpha \{Q_1\}$	
(h-cond)	$\{T \wedge P\} \alpha \{Q\} \wedge \{\neg T \wedge P\} \beta \{Q\} \Rightarrow$	$\{P\} \text{ if } T \text{ then } \alpha \text{ else } \beta \{Q\}$	
(h-while)	$\{T \wedge I\} \alpha \{I\} \Rightarrow$	$\{I\} \text{ while } T \text{ do } \alpha \{\neg T \wedge I\}$	
(h-whilei)	$(P \Rightarrow I) \wedge (I \wedge \neg T \Rightarrow Q) \wedge \{I \wedge T\} \alpha \{I\} \Rightarrow$	$\{P\} \text{ while } T \text{ do } \alpha \text{ inv } I \{Q\}$	
(h-loopi)	$(P \Rightarrow I) \wedge (I \Rightarrow Q) \wedge \{I\} \alpha \{I\} \Rightarrow$	$\{P\} \text{ loop } \alpha \text{ inv } I \{Q\}$	
(h-evoli)	$(P \Rightarrow I) \wedge (I \wedge G \Rightarrow Q) \wedge \{I\} (x' = f \& G)_U^{f_0} \{I\} \Rightarrow$	$\{P\} (x' = f \& G)_U^{f_0} \text{ inv } I \{Q\}$	
(h-conji)	$\{I\} \alpha \{I\} \wedge \{J\} \alpha \{J\} \Rightarrow$	$\{I \wedge J\} \alpha \{I \wedge J\}$	
(h-disji)	$\{I\} \alpha \{I\} \wedge \{J\} \alpha \{J\} \Rightarrow$	$\{I \vee J\} \alpha \{I \vee J\}$	

where $\alpha \text{ inv } I$ is simply α with the annotated invariant I and it binds less than any other program operator, e.g. $\{P\} \text{ loop } \alpha \text{ inv } I \{Q\} = \{P\} (\text{loop } \alpha) \text{ inv } I \{Q\}$.

For automating VCG, the wlp -laws are preferable over the Hoare-style rules since the laws can be added to the proof assistant's simplifier which rewrites them automatically. However, when loops and ODEs are involved, we use the rules (h-whilei), (h-loopi) and (h-evoli). In particular, two workflows emerge for discharging ODEs. If Picard–Lindelöf holds, that is, if there is a unique solution to the system of ODEs and it is known, the law (wlp-flow) is the best choice. Otherwise, we employ the rule (h-evoli) if an invariant is known. See Sect. 6.2 for a procedure guaranteeing the existence of flows or Sect. 5.3 for a procedure determining invariance for evolution commands.

Example 3 We prove that $I s \Leftrightarrow get_g s \geq 0$, or simply $g \geq 0$, is an invariant for the program `blood_sugar = loop (ctrl ; dyn)` from Example 2. That is, we show that $\{I\} \text{ blood_sugar } \{I\}$. We start applying (h-loopi) and proceed with wlp -laws:

$$\begin{aligned}
 & \{I\} \text{ loop } (\text{ctrl} ; \text{dyn}) \text{ inv } I \{I\} \\
 & \Leftrightarrow (I \Rightarrow I) \wedge (I \Rightarrow [\text{ctrl} ; \text{dyn}] I) \wedge (I \Rightarrow I) \\
 & = (\forall s. I s \Rightarrow [\text{if } g \leq g_m \text{ then } g := g_M \text{ else skip}] |g' = -g| I s) \\
 & = (\forall s. I s \Rightarrow (g \leq g_m \Rightarrow |g := g_M| |g' = -g| I s) \wedge (g > g_m \Rightarrow |g' = -g| I s)),
 \end{aligned}$$

where the first equality applies ([wlp-seq](#)) and unfolds the definition of `ctrl` and `dyn`. The second follows by ([wlp-cond](#)). Next, given that $\varphi t c = c \cdot e^{-t}$ is the flow for $g' = -g$:

$$|g' = -g| I s \Leftrightarrow (\forall t \geq 0. I[\varphi t c / g]) \Leftrightarrow (\forall t \geq 0. g \cdot e^{-t} \geq 0) \Leftrightarrow g \geq 0,$$

for all $s \in \mathcal{S}$ by ([wlp-flow](#)), the lens laws, and because $G = \top$ and $k \cdot e^{-t} \geq 0 \Leftrightarrow k \geq 0$. Thus, the conjuncts above simplify to

$$\begin{aligned} g \leq g_m &\Rightarrow (|g' = -g| I)[g_M / g] & g > g_m &\Rightarrow |g' = -g| I s \\ &= g \leq g_m \Rightarrow (g \geq 0)[g_M / g] & &= g > g_m \Rightarrow g \geq 0 \\ &= (g \leq g_m \Rightarrow g_M \geq 0) = \top, & &= \top, \end{aligned}$$

by ([wlp-assig](#)), and because $g_m, g_M \geq 0$. Thus, $(I \Rightarrow |\text{ctrl}; \text{dyn}| I) = \top$. $\square$

3.5.3 Forward Diamonds

Here, we extend our VCG approach by including forward diamonds in our verification framework. Our VCG laws from Sects. 3.5.1 and 3.5.2 help users prove partial correctness specifications. Yet, our approach is generic and extensible and can cover other types of specifications [30, 50, 70]. We have added the *forward diamond* $|\alpha\rangle Q$ – predicate transformer,

$$|\alpha\rangle Q s \Leftrightarrow (\exists s'. s' \in \alpha s \wedge Q s'),$$

that holds if there is a Q -satisfying state of α reachable from s . Due to their semantics, forward diamonds enable us to reason about progress and reachability properties. In applications, this implies that our tool supports proofs describing worst and best case scenarios stating that the modelled system can evolve into an undesired/desired state. See Sect. 8 for an example showing progress for a dynamical system. We formalise and prove the forward diamonds laws below which are also direct consequences of the duality law $|\alpha\rangle Q = \neg |\alpha\rangle \neg Q$. The example after them merely illustrates their seamless application.

(fdia-skip)	$ \text{skip}\rangle Q$	$= Q$	
(fdia-abort)	$ \text{abort}\rangle Q$	$= \perp$	
(fdia-test)	$ _i P?\rangle Q$	$= P \wedge Q$	
(fdia-assign)	$ x := e\rangle Q$	$= Q[e/x]$	
(fdia-seq)	$ \alpha ; \beta\rangle Q$	$= \alpha\rangle \beta\rangle Q$	
(fdia-choice)	$ \alpha \sqcap \beta\rangle Q$	$= \alpha\rangle Q \vee \beta\rangle Q$	
(fdia-loop)	$ \text{loop } \alpha\rangle Q$	$= \exists n. \alpha^n\rangle Q$	
(fdia-cond)	$ \text{if } T \text{ then } \alpha \text{ else } \beta\rangle Q$	$= (T \wedge \alpha\rangle Q) \vee (\neg T \wedge \beta\rangle Q).$	

Additionally, the (informal) diamond law for evolution commands is

$$(\text{fdia-evol}) \quad |(x' = f \ \& \ G)_{U'}^{t_0}\rangle Q s \Leftrightarrow \begin{aligned} &\exists X \in \text{ivp-sols } U \ f \ t_0 s. \exists t \in U s. \\ &(\forall \tau \in t \downarrow_{U s}. G(X \tau)) \Rightarrow Q(X t). \end{aligned} \quad \text{robot icon}$$

and the corresponding law for flows $\varphi : T \rightarrow \mathcal{C} \rightarrow \mathcal{C}$ of f and $U = T = \mathbb{R}_{\geq 0}$ is

$$(\text{fdia-flow}) \quad |x' = f \ \& \ G\rangle Q s \Leftrightarrow (\exists t \geq 0. (\forall \tau \in [0, t]. G(\varphi_s \tau)) \Rightarrow Q(\varphi_s t)). \quad \text{robot icon}$$

Example 4 A similar argument as that in Example 3 allows us to prove the inequality $I \Rightarrow |\text{blood_sugar}| I$ where $I s \Leftrightarrow (g \geq 0)$ and $\text{blood_sugar} = \text{loop } (\text{ctrl}; \text{dyn})$. Namely, we observe that by (fdia-evol), the forward diamond of $g' = -g$ and I becomes

$$|g' = -g| I s \Leftrightarrow (\exists t \geq 0. (I[g \cdot e^{-t}/g])) \Leftrightarrow (\exists t \geq 0. g \cdot e^{-t} \geq 0) \Leftrightarrow g \geq 0,$$

for all $s \in \mathcal{S}$. Hence, the conjuncts below simplify as shown:

$$\begin{aligned} g \leq g_m \wedge (|g' = -g| I)[g_M/g] & \quad g_s > g_m \wedge |g' = -g| I s \\ = g \leq g_m \wedge (g \geq 0)[g_M/g] & \quad = g_s > g_m \wedge g_s \geq 0 \\ = g \leq g_m \wedge g_M \geq 0 = g \leq g_m, & \quad = g_s > g_m. \end{aligned}$$

Therefore, by backward reasoning with the diamond laws, we have

$$\begin{aligned} I & \Rightarrow |\text{loop } (\text{ctrl}; \text{dyn}) \text{ inv } I| I \\ & \Leftarrow I \Rightarrow |\text{ctrl}; \text{dyn}| I \\ & = (\forall s. I s \Rightarrow |\text{if } g \leq g_m \text{ then } g := g_M \text{ else skip}| g' = -g| I s) \\ & = (\forall s. I s \Rightarrow (g \leq g_m \wedge |g := g_M| g' = -g| I s) \vee (g > g_m \wedge |g' = -g| I s)) \\ & = (\forall s. I s \Rightarrow g \leq g_m \vee g > g_m) = \top, \end{aligned}$$

where the first implication follows by a rule analogous to (h-loopi) for diamonds. 

Thus, we have shown that $I \Rightarrow |\text{blood_sugar}| I$. □

We have summarised our approach to hybrid systems verification in general purpose proof assistants [21, 50]. This is the basis for describing our contributions for the rest of this article, included among them, the formalisation of forward diamonds into IsaVODEs. Although a similar formalisation has been done before [70], our implementation is more automated due to its use of standard types, e.g. Isabelle predicates ($\mathcal{S} \rightarrow \mathbb{B}$), that have had more support over time. Thus, our formalisation increases the proof capabilities of IsaVODEs and its expressivity, since the forward diamonds enable us to assert the progress of hybrid programs. Other extensions to our framework not described here are the addition of dL's nondeterministic assignments and their corresponding partial correctness and progress laws, as well as the formalisation of variant-based rules on the reachability of finite iterations and while-loops. See previous work [7, 30] for examples with while-loops that our verification framework could tackle. 

4 Hybrid Modelling Language

Here, we describe our implementation of a hybrid modelling language, which takes advantage of lenses and Isabelle's flexible syntax processing features. Beyond the advantages already mentioned, lenses enhance our hybrid store models in several ways. They allow us to model frames—sets of mutable variables—and thus support local reasoning. They also allow us to project parts of the global store onto vector spaces to describe continuous dynamics. These projections can be constructed using three lens combinators: composition, sum and quotient.

The projections particularly allow us to use hybrid state spaces, consisting of both continuous components with a topological structure (e.g. $\mathbb{R}^n$), and discrete components using Isabelle's flexible type system. This allows our tool to support more general software engineering notations, which typically make use of object-oriented data structures [48]. Moreover,

the projections allow us to reason locally about the continuous variables, since discrete variables are outside of the frame during continuous evolution.

4.1 Dataspaces

Most modelling and programming languages support modules with local variables, constant declarations, and assumptions. We have implemented an Isabelle command that automates the creation of hybrid stores, which provide a local theory context for hybrid programs. We call these *dataspaces*, since they are state spaces that can make use of rich data structures in the program variables. 

```
dataspace store = [parent_store +]
constants  $c_1 :: C_1 \dots c_n :: C_n$ 
assumes  $a_1 : P_1 \dots a_n : P_n$ 
variables  $x_1 :: T_1 \dots x_n :: T_n$ 
```

A dataspace has constants $c_i : C_i$, named constraints $a_i : P_i$ and state variables $x_i : T_i$. In its context, we can create local definitions, theorems and proofs, which are hidden, but accessible using its namespace. Internally, the dataspace command creates an Isabelle **locale** with fixed constants and assumptions, inspired by previous work by Schirmer and Wenzel [66]. Like locales, dataspaces support a form of inheritance, whereby constants, assumptions, and variables can be imported from an existing dataspace (e.g. `parent_store`) and extended with further constants, assumptions, and variables.

Each declared state variable is assigned a lens $x_i :: T_i \Rightarrow \mathcal{S}$, using the abstract store type $\mathcal{S}$ with the lens axioms from Section 3.3 as locale assumptions. We also generate independence assumptions, e.g. $x_i \bowtie x_j$ for $x_i \neq x_j$, that distinguish different variables semantically [25]. Formally, $x \bowtie y$ if $put_x u \circ put_y v = put_y v \circ put_x u$ for all $u, v \in \mathcal{V}$. That is, two lenses are *independent* if their *put* operations commute on all states.

4.2 Lifted Expressions

As discussed in Sect. 3.3, expressions in our hybrid modelling language are modelled by functions of type $\mathcal{S} \rightarrow \mathcal{V}$. Assertions are therefore state predicates, or “expressions” where $\mathcal{V} = \mathbb{B}$. Discharging VCs requires showing that assertions hold for all states. For example, the law (**wlp-test**) requires us to prove a VC of the form $P \Rightarrow Q$, that is, $\forall s. P s \Rightarrow Q s$. Also, if we have state variables x and y , then proving the assertion $x + y \geq x$ corresponds to proving the HOL predicate $get_x s + get_y s \geq get_x s$ for some arbitrary-but-fixed state s , which can readily be discharged using one of Isabelle’s proof methods (**simp**, **auto** etc.). We automate this process via our methods **expr-simp** and **expr-auto**.

Nevertheless, there remains a gap between the syntax used in typical programming languages and its semantic representation. Namely, users would prefer writing $x^2 + y^2 \leq c^2$ over $\lambda s. (get_x s)^2 + (get_y s)^2 \leq c^2$, and so, the main technical challenge is to seamlessly transform between the two. This can be achieved using Isabelle’s syntax pipeline, which significantly improves the usability of our tool.

Isabelle’s multi-stage syntax pipeline parses Unicode strings and transforms them into “pre-terms” [39]: elements of the ML `term` type containing syntactic constants. These must be mapped to previously defined semantic constants by syntax translations, before they can be checked and certified in the Isabelle kernel. Printing reverses this pipeline, mapping terms to strings.

We automate the translation between the expression syntax (pre-terms) and semantics using parse and print translations implemented in Isabelle/ML, as part of our **Shallow-Expressions** component. It lifts pre-terms by replacement of free variables and constants, and insertion of store variables (s) and λ -binders. Its implementation uses the syntactic annotation $(t)_e$ to lift the syntactic term t to a semantic expression in the syntax translation rules. The syntax translation is described by the following equations:

$$(t)_e \equiv [(t)^e]_e, \quad (n)^e \equiv \begin{cases} \lambda s. \text{get}_n s & \text{if } n \text{ is a lens,} \\ \lambda s. n s & \text{if } n \text{ is an expression,} \\ \lambda s. n & \text{otherwise,} \end{cases} \quad (f t)^e \equiv \lambda s. f ((t)^e s),$$

where $p \equiv q$ means that pre-term p is translated to term q , and q printed as p . Moreover, $[-]_e$ is a constant that marks lifted expressions that are embedded in terms. When the translation encounters a name n (i.e. a free variable or constant), it checks whether n has a definition in the context. If it is a lens (i.e. $n::\mathcal{V} \Rightarrow \mathcal{S}$), then it inserts a *get*. If it is an expression (i.e. $n : \mathcal{S} \rightarrow \mathcal{V}$), then it is applied to the state. Otherwise, it leaves the name unchanged, assuming it to be a constant. Function applications are also left unchanged by $\equiv$. For instance, $((x + y)^2 / z)_e \equiv [\lambda s. (\text{get}_x s + \text{get}_y s)^2 / z]_e$ for variables (lenses) x and y and constant z . Once an expression has been processed, the resulting λ -term is enclosed in $[-]_e$. The pretty printer can then recognise a lifted term and print it. This process is fully automated, so that users see only the sugared expression, without the λ -binders, in both the parser and terms' output during the proving process. 

4.3 Substitutions

In our semantic approach, substitutions correspond to functions $\sigma : \mathcal{S} \rightarrow \mathcal{S}$. We can then denote updates as sequences of variable assignments. That is, instead of directly manipulating the store $s : \mathcal{S}$ with the lens functions, we provide user-friendly program specifications with the notation $\sigma(x \rightsquigarrow e) = \lambda s. \text{put}_x(e s)(\sigma s)$. Thus, assignments can be described as sequences of updates for lenses $x_i::\mathcal{V}_i \Rightarrow \mathcal{S}$ and “expressions” $e_i : \mathcal{S} \rightarrow \mathcal{V}_i$:

$$[x_1 \rightsquigarrow e_1, x_2 \rightsquigarrow e_2, \dots] = id(x_1 \rightsquigarrow e_1)(x_2 \rightsquigarrow e_2) \dots \quad (\text{lens-subs})$$

Implicitly, any variable y not mentioned in such a substitution is left unchanged: $y \rightsquigarrow y$. We further write $e[v/x] = e \circ [x \rightsquigarrow v]$, with $x::\mathcal{V} \Rightarrow \mathcal{S}$, $e : \mathcal{S} \rightarrow \mathcal{V}'$, and $v : \mathcal{S} \rightarrow \mathcal{V}$, for the application of substitutions to expressions. This yields standard notations for program specifications, e.g. $(x := e) = \langle [x \rightsquigarrow e] \rangle$ and $wlp \langle [x \rightsquigarrow e] \rangle Q = Q[e/x]$. Using an Isabelle simplification procedure (a “simproc”), the simplifier can reorder assignments alphabetically according to their variable name, and otherwise reduce and evaluate substitutions during VCG [25]. We can extract assignments for x writing $\langle \sigma \rangle_s x = \text{get}_x \circ \sigma$ so that, e.g. $\langle [x \rightsquigarrow e_1, y \rightsquigarrow e_2] \rangle_s x$ reduces to e_1 when $x \bowtie y$. 

Example 5 We continue our blood glucose running example and formalise Example 2. We declare our problem variables and assumptions via our **dataspace** command, name this dataspace `glucose`, and assume that there is a minimal warning threshold $g_m > 0$ and a maximum dosage $g_M > g_m$. The “continuous” variable g represents the patient’s glucose.

```
dataspace glucose =
  constants g_m :: real g_M :: real
```

```

assumes ge_0: "gm > 0" and ge_gm: "gM > gm"
variables g :: real

```

Next, inside the glucose context we declare, via Isabelle’s **abbreviation** command, the definition of the controller and the dynamics. Our shallow expressions hide the lens infrastructure and, from the user’s perspective, the definitions are Isabelle abbreviations. Notice also, that our recently introduced “substitution” notation allows us to explicitly specify the flow’s behaviour on the continuous variable g . It also occurs implicitly in our declaration of the differential equation $g' = g$ (see Sect. 5.2).

```

context glucose
begin

abbreviation "ctrl  $\equiv$  IF  $g \leq g_m$  THEN  $g ::= g_M$  ELSE skip"

abbreviation "dyn  $\equiv$  { $g' = -g$ }"

abbreviation "flow  $\tau \equiv$  [ $g \rightsquigarrow g * \exp(-\tau)$ ]"

abbreviation "blood_sugar  $\equiv$  LOOP (ctrl; dyn) INV ( $g \geq 0$ )"

end

```

Thus, our lens integrations provide a seamless way to formalise hybrid system verification problems in Isabelle. We explore their verification condition generation in Sect. 6.2. $\square$

4.4 Vectors and Matrices

Vectors and matrices are ubiquitous in engineering applications and users of our framework would appreciate using familiar concepts and notations to them. This is possible due to our modelling language. In particular, vectors are supported by HOL-Analysis using finite Cartesian product types, (A, n) **vec** with the notation A^n . Here, A is the element type, and n is a numeral type denoting the dimension. The type of vectors is isomorphic to $[n] \rightarrow A$ where $[n] = \{1, \dots, n\}$. A matrix is simply a vector of vectors, $A^{m \times n}$, hence a map $[m] \rightarrow [n] \rightarrow A$. Building on this, we supply notation $[[x_{11}, \dots, x_{1n}], \dots, [x_{m1}, \dots, x_{mn}]]$ for matrices and means for accessing coordinates of vectors via hybrid program variables [19]. This notation supports the inference of vector and matrices’ dimensions conveyed by the type variables.

Vectors and matrices are often represented as composite objects consisting of several values, e.g. $p = (p_x, p_y) \in \mathbb{R}^2$. When writing specifications, it is often convenient to refer to and manipulate these components individually. We can denote such variables using component lenses and the lens composition operator. We write $x_1 \circ x_2 :: \mathcal{S}_1 \Rightarrow \mathcal{S}_3$, for $x_1 :: \mathcal{S}_1 \Rightarrow \mathcal{S}_2$, $x_2 :: \mathcal{S}_2 \Rightarrow \mathcal{S}_3$, for the forward composition and $1_{\mathcal{S}} :: \mathcal{S} \Rightarrow \mathcal{S}$ for the units in the lens category, but do not show formal definitions [25]. Intuitively, the composition $\circ$ selects part of a larger store as illustrated below.

We model vectors in $\mathbb{R}^n$ as part of larger hybrid stores, lenses $v :: \mathbb{R}^n \Rightarrow \mathcal{S}$, and project onto coordinate $v_k :: \mathbb{R} \Rightarrow \mathcal{S}$ using lens composition and a *vector lens* $\Pi(i) :: \mathbb{R} \Rightarrow \mathbb{R}^n$: 🧠

$$\begin{aligned}
 \Pi(i) &= (\text{get}_{\Pi(i)}, \text{put}_{\Pi(i)}), \text{ where} \\
 \text{get}_{\Pi(i)} &= (\lambda s. \text{vec-nth } s \ i), \\
 \text{put}_{\Pi(i)} &= (\lambda v s. \text{vec-upd } s \ i \ v),
 \end{aligned}$$

and $i \in [n] = \{1, \dots, n\}$. The lookup function $vec\text{-}nth : A^n \rightarrow [n] \rightarrow A$ and update function $vec\text{-}upd : A^n \rightarrow [n] \rightarrow A \rightarrow A^n$ come from HOL-Analysis and satisfy the lens axioms (lens-laws). Then, as an example, $p_x = \Pi(1) \circ p$ and $p_y = \Pi(2) \circ p$ for $p : \mathbb{R}^2 \Rightarrow \mathcal{S}$, using $\circ$ to first select the variable p and then the vector-part of the hybrid store. Intuitively, two vector elements are independent, $\Pi(i) \bowtie \Pi(j)$ iff they have different indices, $i \neq j$.

Example 6 To illustrate the use of vector variables, we model the dynamics and a controller for an autonomous boat. We refer readers to previous publications for the verification of an invariant for this system [19, 21]. The boat is manoeuvrable in $\mathbb{R}^2$ and has a rotatable thruster generating a positive propulsive force f with maximum f_{max} . The boat's state is determined by its position $p = (p_x, p_y)$, velocity $v = (v_x, v_y)$, and acceleration $a = (a_x, a_y)$. We describe this state with the following dataspace:

```
dataspace AMV =
  constants S::ℝ f_max::ℝ assumes f_max:"f_max ≥ 0"
  variables p::"ℝ vec[2]" v::"ℝ vec[2]" a::"ℝ vec[2]" φ::ℝ s::ℝ
  wps::"(ℝ vec[2]) list" org::"(ℝ vec[2]) set" rs::ℝ rh::ℝ
```

This store model combines discrete and continuous variables and uses the alternative notation $\mathbb{R} \text{ vec}[n]$ for a real-valued vector of dimension n . The **dataspace** specifies a variable for linear speed s , and a constant S for the boat's maximum speed. We also provide the discrete variable **wps** for a list of points to pass through in the vehicle's path (way-point path), **org** for a set of points where obstacles are located (obstacle register), and the requested speed and heading (**rs** and **rh**). Our **dataspace** allows us to declare variables $p, v, a : \mathbb{R} \text{ vec}[2]$ and manipulate them using operations for vectors (see Sect. 5.2). $\square$

5 Local Reasoning

In this section, we describe our framework's support for local reasoning, which allows us to consider only parts of the state that are changed by a component in the verification. This improves the scalability of our approach, since we can decompose verification tasks into smaller manageable tasks, in an analogous way to separation logic [63]. We provide our main theoretical contributions here. That is, we show how lenses can be used to characterise a program's frame: the set of variables which may be modified. We then explain how frames extend to evolution commands, such that variables with no derivative (or derivative 0) are outside of the frame. Next, we develop a framed version of differentiation, called *framed Fréchet derivatives*, which allows us to perform local differentiation with respect to a strict subset of the store variables. This, in turn, supports a method, framed differential induction, for proving invariants in the continuous part of the state space. Finally, we introduce a corresponding implementation of dL's differential ghost rule [61] that augments systems of ODEs with fresh equations to aid invariant reasoning. This rule likewise supports frames.

5.1 Frames

Lenses support algebraic manipulations of variable frames. A *frame* is the set of variables that a program is permitted to change. Variables outside of the frame are immutable. We first show how variable sets can be modelled via lens sums. Then we recall a predicate characterising immutable program variables [22]. Most importantly, we derive a frame rule à la separation logic for local reasoning with framed variables.

Variable lenses $x_1::\mathcal{V}_1 \Rightarrow S$ and $x_2::\mathcal{V}_2 \Rightarrow S$ can be combined into lenses for variable sets with *lens sum* [25], $x_1 \oplus x_2::\mathcal{V}_1 \times \mathcal{V}_2 \Rightarrow S$ if $x_1 \bowtie x_2$ via

$$\begin{aligned} \text{get}_{x_1 \oplus x_2}(s_1, s_2) &= (\text{get}_{x_1} s_1, \text{get}_{x_2} s_2), \text{ and} \\ \text{put}_{x_1 \oplus x_2}(v_1, v_2) &= \text{put}_{x_1} v_1 \circ \text{put}_{x_2} v_2. \end{aligned}$$

This combines two independent lenses into a single lens with a product view. It can be used to model composite variables, for example, $(x \oplus y) := (e, f)$ is a simultaneous assignment to x and y . We can decompose such a composite update into two atomic updates, with $[(x, y) \rightsquigarrow (e_1, e_2)] = [x \rightsquigarrow e_1, y \rightsquigarrow e_2]$. We can also use lens sums to model finite sets, for example $\{x, y, z\}$ is modelled as $x \oplus (y \oplus z)$. Each variable in such a sum may have a different type, e.g. $\{v_x, \vec{p}\}$ is a valid and well-typed construction. 

Lens sums are only associative and commutative up-to isomorphism of cartesian products. We need heterogeneous orderings and equivalences between lenses to capture this. We define a *lens preorder* [25], $x_1 \leq x_2 \Leftrightarrow \exists x_3. x_1 = x_3 \circ x_2$ that captures the part-of relation between $x_1::\mathcal{V}_1 \Rightarrow S$ and $x_2::\mathcal{V}_2 \Rightarrow S$, e.g. $v_x \leq \vec{v}$ and $\vec{p} \leq \vec{p} \oplus \vec{v}$. *Lens equivalence* $\cong = \leq \cap \geq$ then identifies lenses with the same shape in the store. Then, for variable set lenses up-to $\cong$, $\oplus$ models $\cup$, $\bowtie$ models $\Join$,

and $\leq$ models $\subseteq$ or $\in$. Since $x_1 \leq x_1 \oplus x_2$ and $x_1 \oplus x_2 \cong x_2 \oplus x_1$, with our variable set interpretation, we can show, e.g., that $x \in \{x, y, z\}$, $\{x, y\} \subseteq \{x, y, z\}$, and $\{x, y\} = \{y, x\}$. Hence we can use these lens combinators to construct and reason about variable frames. 

We can use variable set lenses to capture the frame of a program. Let $A::\mathcal{V} \Rightarrow S$ be a lens modelling a variable set. For $s_1, s_2 \in S$ let $s_1 \approx_A s_2$ hold if $s_1 = s_2$ up-to the values of variables in A , that is $\text{get}_A s_1 = \text{get}_A s_2$. Local reasoning within A uses the *lens quotient* [18] $x \parallel A$, which localises a lens $x::\mathcal{V} \Rightarrow S$ to a lens $\mathcal{V} \Rightarrow C$. Assuming $x \leq A$, it yields $x_1::\mathcal{V} \Rightarrow C$ such that $x = x_1 \circ A$. For example, $p_x \parallel p = \Pi(1)$ with $C = \mathbb{R}^n$. 

We can also use lenses to describe when a variable does not occur freely in an expression or predicate with the unrestriction property: $A \nmid e \Leftrightarrow \forall v. e \circ (\text{put}_A v) = e$ [25]. A variable x is unrestricted in e , written $x \nmid e$, provided that e does not semantically depend on x for its evaluation. For example, $x \nmid (y + 1)$, when $x \bowtie y$, since $y + 1$ does not mention x . We also define $(-A) \nmid e \Leftrightarrow \forall s_1 s_2 v. e(\text{put}_A v s_1) = e(\text{put}_A v s_2)$ as the converse, which requires that e does not depend on variables outside of A .

Non-modification Next, we capture the non-modification of variables by a program. For $\alpha : S \rightarrow \mathcal{P}S$ and an expression (or predicate) e we define $\alpha \text{ nmods } e \Leftrightarrow (\forall s_1 \in S. e(s_1) = e(s_2))$, which describes when e does not depend on the mutable variable of α . The expression e can characterise a set of variables giving the set of immutable variables. For example, we have it that $(x := x + 1) \text{ nmods } (y, z)$, when $x \bowtie y$ and $x \bowtie z$, since this assignment changes only x and no other variables.

Intuitively, non-modification $\alpha \text{ nmods } x$, where x is a variable lens, is equivalent to the specification for $\{x = v\} \alpha \{x = v\}$ for fresh logical variable v . This means that x retains its initial value in any final state of α . We prove the following laws for non-modification:

$$\frac{A \nmid x}{(x := e) \text{ nmods } A} \quad \frac{-}{iP? \text{ nmods } A} \quad \frac{\alpha \text{ nmods } A \quad \beta \text{ nmods } A}{(\alpha \circ \beta) \text{ nmods } A} \quad \frac{\alpha \text{ nmods } A \quad \beta \text{ nmods } A}{(\alpha \bowtie \beta) \text{ nmods } A} \quad \frac{\alpha \text{ nmods } A \quad \beta \text{ nmods } A}{\alpha^* \text{ nmods } A} \quad \frac{\alpha \text{ nmods } B \quad A \leq B}{\alpha \text{ nmods } A}$$

The variables in A are immutable for assignment $x := e$ provided x is not in A . A test $iP?$ modifies no variables, and therefore any set A is immutable. For the programming operators,

non-modification is inherited from the parts. The final law shows that we can always shrink the specified set of immutable variables.

With these concepts in place, we derive two frame rules for local reasoning:



$$\frac{\alpha \text{ nmods } I \quad \{P\} \alpha \{Q\}}{\{P \wedge I\} \alpha \{Q \wedge I\}} \quad \frac{\alpha \text{ nmods } A \quad (-A) \nmid I \quad \{P\} \alpha \{Q\}}{\{P \wedge I\} \alpha \{Q \wedge I\}}$$

If program α does not modify any variables mentioned in I , then I can be added as an invariant of α . In the first law, non-modification is checked directly of the variables used by I . In the second, which is an instance of the first, we instead infer the immutable variables of A and check that I does not depend on variables outside of A . With these laws, we can import invariants for a program fragment that refer to only those variables that are left unchanged. This allows us to perform modular verification, whereby we need only consider invariants of variables used in a component. In the following section, we show how this can be applied to systems of ODEs.

5.2 Framed Evolution Commands

We extend previous components [50] for continuous dynamics with function framing techniques that project onto parts of the store. That is, we formally describe the implementation of the evolution command state transformer using the lens infrastructure described so far [21]. Specifically, we use framing to derive continuous vector fields ($\mathcal{C} \rightarrow \mathcal{C}$) and flows from state-wide “substitutions” ($\mathcal{S} \rightarrow \mathcal{S}$). We also add a non-modification rule for evolution commands. This supports local reasoning where evolution commands modify only continuous variables and leave discrete ones—outside a frame—unchanged.

Framing uses the second interpretation of lenses where the frame $\mathcal{C}$ is a subregion of $\mathcal{S}$ that we can access through $x::\mathcal{C} \Rightarrow \mathcal{S}$. We view the store as divided into its continuous $\mathcal{C}$ and discrete parts and localise continuous variables to the former. The continuous part must have sufficient topological structure to support derivatives and is thus restricted to certain type constructions like normed vector spaces or the real numbers. However, the discrete part may use any type defined in HOL. With this view, we can use get_x and put_x to lift entities defined on $\mathcal{C}$ or project those in $\mathcal{S}$. For instance, given any $s \in \mathcal{S}$ and a predicate $G : \mathcal{S} \rightarrow \mathbb{B}$ (like the guards in evolution commands), there is a corresponding restriction $G \downarrow_x^s : \mathcal{C} \rightarrow \mathbb{B}$ such that $G \downarrow_x^s \vec{c} \Leftrightarrow G[\vec{c}/x] \Leftrightarrow G(put_x \vec{c} s)$. Conversely, for $s \in \mathcal{S}$ and X , a set of vectors in $\mathcal{C}$, the set $X \uparrow_x^s = \mathcal{P}(\lambda \vec{c}. put_x \vec{c} s) X$ has values in $\mathcal{S}$.

More importantly, we can specify ODEs and flows via time-dependent deterministic functions (Sect. 4.3’s **lens-subs**). Given a lens $x::\mathcal{C} \Rightarrow \mathcal{S}$ from global store $\mathcal{S}$ onto local continuous store $\mathcal{C}$ and $s \in \mathcal{S}$, we can turn any state-wide function $f : \mathcal{T} \rightarrow \mathcal{S} \rightarrow \mathcal{S}$ into a vector field $f \downarrow_x^s : \mathcal{T} \rightarrow \mathcal{C} \rightarrow \mathcal{C}$ by framing it via $f \downarrow_x^s t \vec{c} = get_x((f t)(put_x \vec{c} s))$.

Example 7 Suppose $\mathcal{S} = \mathbb{R}^2 \times \mathbb{R}^2 \times \mathbb{R}^2 \times \mathcal{S}'$ and $p, v, a::\mathbb{R}^2 \Rightarrow \mathcal{S}$. The variable set lens $A = (p \oplus v \oplus a)::\mathbb{R}^2 \times \mathbb{R}^2 \times \mathbb{R}^2 \Rightarrow \mathcal{S}$ frames the continuous part of the state space $\mathcal{S}$. The substitution $f : \mathcal{T} \rightarrow \mathcal{S} \rightarrow \mathcal{S}$ such that $f t = [p \rightsquigarrow v, v \rightsquigarrow a, a \rightsquigarrow 0]$ then behaves as the identity on $\mathcal{S}'$ and becomes the vector field $f \downarrow_A^s : \mathcal{T} \rightarrow \mathbb{R}^2 \times \mathbb{R}^2 \times \mathbb{R}^2 \rightarrow \mathbb{R}^2 \times \mathbb{R}^2 \times \mathbb{R}^2$. Hence, f naturally describes the ODEs $p' t = v t$, $v' t = a t$, $a' t = 0$ after framing. $\square$

Using the previously described liftings and projections, we formally define the semantics of evolution commands. For this, we only need to lift the definition of generalised guarded orbits maps (**orbit-map** in Sect. 3.4) on the continuous $\mathcal{C}$ to the larger space $\mathcal{S}$. Thus, for

substitution $f : T \rightarrow S \rightarrow S$, predicate $G : S \rightarrow \mathbb{B}$, interval function $U : \mathcal{C} \rightarrow \mathcal{P} \mathbb{R}$, and $t_0 \in \mathbb{R}$ the state transformer $S \rightarrow \mathcal{P} S$ modelling evolution commands is

$$(x' = f \& G)_U^{t_0} s = (\gamma^{f \downarrow_x^s} (G \downarrow_x^s) U t_0 x_s) \uparrow_x^s, \text{ or equivalently}$$

$$(x' = f \& G)_U^{t_0} s = \left\{ put_x (X t) s \mid \left\| \begin{array}{l} t \in U x_s \wedge X \in ivp\text{-}sols U (f \downarrow_x^s) t_0 x_s \\ \wedge \mathcal{P} X (t \downarrow_{U x_s}) \subseteq G \downarrow_x^s \end{array} \right\| \right\},$$

where we abbreviate $get_x s$ with x_s . That is, evolution commands are state transformers that output those states whose discrete part remains unchanged from s but whose continuous part changes according to the ODEs' solutions within G . With this, the law (**wlp-evol**) formally becomes

$$|(x' = f \& G)_U^{t_0} Q s \Leftrightarrow \forall X \in ivp\text{-}sols U (f \downarrow_x^s) t_0 x_s. \forall t \in U x_s. (\forall \tau \in t \downarrow_{U x_s}. G[X \tau/x]) \Rightarrow Q[X t/x].$$

This says that the postcondition Q holds after an evolution command $(x' = f \& G)_U^{t_0}$ for $s \in S$ if every solution X to the IVP corresponding to (t_0, x_s) satisfies Q on every time t , provided G holds from the beginning of the interval $t_0 \in U x_s$ until t . Thus, VCG follows our description in Sect. 3: users must supply flows and evidence for Lipschitz continuity in order to obtain *wlps*. We provide tactics that automate these processes in Sect. 6.

We use Isabelle's syntax translations to provide a natural syntax for specifying evolution commands. Users can write $\{x'_1 = e_1, \dots, x'_n = e_n \mid G U V @ t_0\}$ directly into the prover where each $x_i :: \mathcal{V}_i \Rightarrow S$ is a summand of the frame lens $x = \{x_1, \dots, x_n\} :: \mathcal{C} \Rightarrow S$. Users can thus declare the ODEs in evolution commands coordinate-wise with lifted expressions $e_i : S \rightarrow \mathcal{V}_i$. They can also omit the parameters G, U, V and t_0 which defaults them to $\top, \mathbb{R}_{\geq 0}, \mathcal{C}$ and 0, respectively. If desired, they can also use product syntax $(x'_1, \dots, x'_n) = (e_1, \dots, e_n)$ or vector syntax $x' = e$, and specify evolution commands using flows instead of ODEs with the notation $\{\text{EVOL } x = e \tau \mid G\}$.

With these, non-modification of variables naturally extends to ODEs with the law

$$|(x' = f \& G)_U^{t_0} Q s \Leftrightarrow \forall X \in ivp\text{-}sols U (f \downarrow_x^s) t_0 x_s. \forall t \in U x_s. (\forall \tau \in t \downarrow_{U x_s}. G[X \tau/x]) \Rightarrow Q[X t/x].$$

Specifically, any set of variables (A) without assigned derivatives in a system of ODEs is immutable. Then, by application of the frame rule, we can demonstrate that any assertion I that uses only variables outside of x is an invariant of the system of ODEs.

Example 8 We use the autonomous boat from Example 6 to illustrate the use of non-modification. A system of ODEs for the boat's state p, v, a may be specified as follows:

abbreviation "ODE $\equiv \{ p' = v, v' = a, a' = 0, \phi' = \omega,$
 $s' = \text{if } s \neq 0 \text{ then } (v \cdot a) / s \text{ else } \|a\|$
 $\mid s *_R [\sin(\phi), \cos(\phi)] = v \wedge 0 \leq s \wedge s \leq S \}$ "

s for ϕ and s . The derivative of the former is the angular velocity ω , which has the value $\arccos((v+a) \cdot v / (\|v+a\| \cdot \|v\|))$ when $\|v\| \neq 0$ and 0 otherwise [19]. The linear acceleration (s') is calculated using the inner product of v and a . If the current speed is 0, then s' is $\|a\|$. Immediately after the derivatives, we also specify the guard or boundary condition that serves to constrain the relationship between the velocity vector and the heading ϕ . The guard states that the velocity vector v is equal to s multiplied with the heading unit-vector using scalar

multiplication ($\ast R$) and our vector syntax. We also require that $0 \leq s \leq S$, i.e. that the linear speed is between 0 and the maximum speed.

All other variables in the store remain outside the evolution frame and do not need to be specified. In particular, notice that the *ODE* above does not mention the requested speed variable rs . This is a discrete variable that is unchanged during evolution. Therefore, we can show: $ODE n mods\ rs$. Moreover, using the frame rule we can also demonstrate that $rs > 0$ is an invariant, i.e. $\{rs > 0\} ODE \{rs > 0\}$ [21]. $\square$

5.3 Frames and Invariants for ODEs

As discussed in Sect. 3, an alternative to using flows for verification of evolution commands is finding and certifying invariants for them. Mathematically, evolution commands' invariants coincide with *invariant sets* for dynamical systems or $d\mathcal{L}$'s *differential invariants* [50]. We abbreviate the statement " I is an invariant for the evolution command $(x' = f \ \& \ G)_U^{t_0}$ " with the notation $diff\text{-}inv\ x \ f \ G \ U \ t_0 \ I$. In terms of Hoare logic, invariants for evolution commands satisfy

$$diff\text{-}inv\ x \ f \ G \ U \ t_0 \ I \Leftrightarrow \{I\} (x' = f \ \& \ G)_U^{t_0} \{I\}.$$

Informally, $diff\text{-}inv\ x \ f \ G \ U \ t_0 \ I$ asserts that all states in the generalised guarded orbit $(x' = f \ \& \ G)_U^{t_0} s$ of $s \in \mathcal{S}$ such that $I\ s$, will also satisfy I . In dynamical systems parlance, the orbits of the system of ODEs within the region characterised by I remain within I .

A common approach in hybrid program verification for certifying invariants for evolution commands is *differential induction* [59]. It establishes sufficient conditions for guaranteeing that simple predicates, such as (in)equalities, are invariants. From these, more complex predicates like conjunctions or disjunctions of these (in)equalities can be shown to be invariants using the rules (**h-conji**) and (**h-disji**).

Example 9 To prove that the conjunction $x > c \wedge y \geq x$ is an invariant of the pair of ODEs $x' = 1, y' = 2$ with $c \in \mathbb{R}$ (a constant) we need to show that

$$\{x > c \wedge y \geq x\} (x' = 1, y' = 2) \{x > c \wedge y \geq x\}.$$

An application of the rule (**h-conji**) yields the two proof obligations

$$\{x > c\} (x' = 1, y' = 2) \{x > c\}, \text{ and } \{y \geq x\} (x' = 1, y' = 2) \{y \geq x\}.$$

We conclude the proof informally to provide an intuition for how to proceed:

Since the derivative of x is greater than 0, its magnitude is increasing. Hence, for all time $t \geq 0$, the value of $x\ t$ is greater or equal to its original value $x\ 0 > c$. This means that the "values" of x remain above c . Similarly, since the derivative of y is greater than that of x and positive, y "grows" faster than x . Hence, the value of y remains greater or equal to that of x . Thus, the predicate $x > c \wedge y \geq d$ is an invariant of $x' = 1, y' = 2$. $\square$

Formally, if a predicate I is in negation normal form (NNF) in a first-order language for the real numbers $\mathcal{L}_{\mathbb{R}}(\langle 0, 1, +, -, \cdot, <, \leq \rangle)$, to show that it is an invariant, we can apply the rules (**h-conji**) and (**h-disji**) until the only remaining proof obligations are Hoare triples of literals. The negated literals can also be converted into positive ones via the equivalences $\neg(x < y) \Leftrightarrow y \leq x$, $\neg(x \leq y) \Leftrightarrow y < x$, and $\neg(x = y) \Leftrightarrow (y < x \vee x < y)$. The remaining proof obligations can be discharged by analysing the derivatives of the magnitudes represented in them as done in Example 9. In the sequel, we present the theory to do this analysis formally in our setting.

The discussion in Example 9 compares derivatives of expressions depending on the ODEs' variables. In our semantic approach, the system of ODEs is modelled by a function $f t : S \rightarrow S$ that becomes a vector field ($\mathcal{C} \rightarrow \mathcal{C}$) after framing via some $x :: \mathcal{C} \Rightarrow S$, where $\mathcal{C}$ is the continuous part of S . Similarly, our "expressions" are really functions $e : S \rightarrow \mathcal{U}$ (see Sect. 3.3), and we can assume that $\mathcal{U}$ is a continuous state space to get "continuous expressions". We frame these functions to the continuous part $\mathcal{C}$ of S to obtain our *framed expressions* $e \downarrow_s^x : \mathcal{C} \rightarrow \mathcal{U}$ such that $e \downarrow_s^x = e \circ (\lambda \tilde{c}. put_x \tilde{c} s)$. We wish to "differentiate" these expressions as informally done in Example 9. Hence, for a general and formal treatment of our semantic entities, we use the Fréchet derivative of these framed expressions. More specifically, recall that, if a function $F : \mathcal{C} \rightarrow \mathcal{U}$ between normed spaces $\mathcal{C}, \mathcal{U}$ is Fréchet differentiable at $\tilde{c}$, the Fréchet derivative of F at $\tilde{c}$ is the bounded linear operator $D F \tilde{c} : \mathcal{C} \rightarrow \mathcal{U}$ that attests this. In the finite-dimensional case, e.g. $F : \mathbb{R}^n \rightarrow \mathbb{R}^m$ with $m, n \in \mathbb{N}$, the Fréchet derivative $D F \tilde{c}$ is the Jacobian. It is well-known that if $\tilde{e}_i$ is the i -th unit vector of the canonical ordered base, the function $\lambda \tilde{x}. (D F \tilde{x}) \tilde{e}_i$ provides the i th partial derivatives of F while the directional derivative of F in the direction of $\tilde{c}$ is $\lambda \tilde{x}. (D F \tilde{x}) \tilde{c}$. With these ideas in mind, we define our *framed Fréchet derivatives* $\mathcal{D}_x^f e : S \rightarrow \mathcal{U}$ of expression $e : S \rightarrow \mathcal{U}$ in the direction of $f : S \rightarrow S$ with respect to $x :: \mathcal{C} \Rightarrow S$ as [21]

$$(\mathcal{D}_x^f e) s = (D e \downarrow_s^x (get_x s)) (get_x (f s)).$$

That is, they are the directional derivatives of framed expressions $e \downarrow_s^x$ in the direction of the projection of f onto the continuous space $\mathcal{C}$. These framed Fréchet derivatives capture the intuitive analysis performed in Example 9. In fact, the following rules are sound:

$$\begin{aligned} (\text{dinv-eq}) \quad & (G \Rightarrow \mathcal{D}_x^f e_1 = \mathcal{D}_x^f e_2) \Rightarrow \text{diff-inv } x \ f \ \mathbb{R}_{\geq 0} \ 0 \ (e_1 = e_2) \\ (\text{dinv-leq}) \quad & (G \Rightarrow \mathcal{D}_x^f e_1 \leq \mathcal{D}_x^f e_2) \Rightarrow \text{diff-inv } x \ f \ \mathbb{R}_{\geq 0} \ 0 \ (e_1 \leq e_2) \\ (\text{dinv-less}) \quad & (G \Rightarrow \mathcal{D}_x^f e_1 \leq \mathcal{D}_x^f e_2) \Rightarrow \text{diff-inv } x \ f \ \mathbb{R}_{\geq 0} \ 0 \ (e_1 < e_2) \end{aligned}$$

The rule (dinv-eq) asserts that showing that an equality is an invariant reduces to showing that both sides of the equality change at the same rate over time. Similarly, the rules (dinv-leq) and (dinv-less) state that showing that inequalities are invariants requires showing that the rates of change on both sides preserve or augment the initial difference.

From the users' perspective, $\mathcal{D}_x^f e$ operates as the derivative of expression e with respect to the variables x according to the system of ODEs f . Well-known laws hold.

$$\mathcal{D}_x^f k = 0 \quad \text{if } x \nmid k, \quad (1)$$

$$\mathcal{D}_y^f x = 0 \quad \text{if } x \not\triangleright y, \quad (2)$$

$$\mathcal{D}_X^f x = \langle f \rangle_s x \quad \text{if } x \in X \text{ and } get_{x//X} \text{ is a bounded linear operator}, \quad (3)$$

$$\mathcal{D}_x^f (e_1 + e_2) = (\mathcal{D}_x^f e_1) + (\mathcal{D}_x^f e_2), \quad (4)$$

$$\mathcal{D}_x^f (e_1 \cdot e_2) = (e_1 \cdot \mathcal{D}_x^f e_2) + (\mathcal{D}_x^f e_1 \cdot e_2), \quad (5)$$

$$\mathcal{D}_x^f e^n = n \cdot (\mathcal{D}_x^f e) \cdot e^{(n-1)}, \quad (6)$$

$$\mathcal{D}_x^f \ln(e) = (\mathcal{D}_x^f e) / e \quad \text{if } e > 0. \quad (7)$$

In summary, users can read (1) and (2) as stating that the derivative of constants or variables outside the frame of differentiation are 0. The law (3) says that the derivative of a variable inside the frame is dictated by the ODE, and thus, users simply need to substitute according

to f . The remaining laws are well-known differentiation properties such as linearity of derivatives or the Leibniz rule.

Example 10 First, consider the various Fréchet derivatives of the expression z^2 . Using the ODE $z' = 1$ (below as a [lens-subst](#) $[z \rightsquigarrow 1]$), the resulting expression is

$$\mathcal{D}_z^{[z \rightsquigarrow 1]} z^2 = 2 \cdot (\mathcal{D}_z^{[z \rightsquigarrow 1]} z) \cdot z = 2 \cdot 1 \cdot z = 2z.$$

However, differentiating with respect to a different variable yields

$$\mathcal{D}_y^{[z \rightsquigarrow 1]} z^2 = 2 \cdot (\mathcal{D}_y^{[z \rightsquigarrow 1]} z) \cdot z = 2 \cdot 0 \cdot z = 0,$$

assuming $y \bowtie z$. Finally, the ODE changes the final result

$$\mathcal{D}_z^{[z \rightsquigarrow 2]} z^2 = 2 \cdot (\mathcal{D}_z^{[z \rightsquigarrow 2]} z) \cdot z = 2 \cdot 2 \cdot z = 4z.$$

Therefore, certifying invariants for evolution commands reduces to computing framed Fréchet derivatives and comparing the results which are often easily computable by rewriting. For instance, to certify that $z > 0$ is an invariant of $z' = z^2$, it suffices to check

$$0 = \mathcal{D}_z^{[z \rightsquigarrow z^2]} 0 \leq \mathcal{D}_z^{[z \rightsquigarrow z^2]} z = z^2$$

by rule ([dinv-less](#)). We can now formally culminate the proof in Example 9. By rules ([dinv-less](#)) and ([dinv-leq](#)) respectively, the assertions $x > c$ and $y \geq x$ are invariants since

$$1 = \mathcal{D}_{x \oplus y}^{[x \rightsquigarrow 1, y \rightsquigarrow 2]} x \geq \mathcal{D}_{x \oplus y}^{[x \rightsquigarrow 1, y \rightsquigarrow 2]} c = 0 \text{ and } 2 = \mathcal{D}_{x \oplus y}^{[x \rightsquigarrow 1, y \rightsquigarrow 2]} y \geq \mathcal{D}_{x \oplus y}^{[x \rightsquigarrow 1, y \rightsquigarrow 2]} x = 1.$$

Thus, this example and Example 9 illustrate how to certify invariants through dL's differential induction method by applying our semantic framed (Fréchet) differentiation. $\square$

5.4 Ghosts and Darboux Rules

Differential induction does not suffice to prove all invariant certifications [55, 58]. For instance, applying rule ([dinv-less](#)) to the dynamics $x' = -x$ of Examples 1–4 to show invariance of $x > 0$ does not lead to a concluding proof state. Indeed, $\mathcal{D}_x^{[x \rightsquigarrow -x]} x = -x$ is not necessarily greater or equal to $0 = \mathcal{D}_x^{[x \rightsquigarrow -x]} 0$. For those cases, differential dynamic logic dL includes the *differential ghost* [55] rule which asserts the correctness of an evolution command given the correctness of a higher-dimensional but equivalent system of ODEs. Here we generalise our previous formalisation of this rule [21] and use it to derive dL's three Darboux rules [55]. The latter concretise the ghost rule to the atomic cases of differential induction: (in)equalities. Thus, they help prove invariance of predicates where differential induction does not apply. Concretely, we formalise and prove the soundness of the rules:

$$(dG) \quad \frac{\{P\} \{x' = f, y' = A \cdot y + b \ \& \ G\} \{Q\}}{\{\exists v. P[v/y]\} \{x' = f \ \& \ G\} \{\exists v. Q[v/y]\}}, \quad \text{👤}$$

$$(dbx\text{-eq}) \quad \frac{\mathcal{D}_{x+Ly}^{f(y \rightsquigarrow -c \cdot y)} e = c \cdot e}{\{e = 0\} \{x' = f \ \& \ G\} \{e = 0\}}, \quad \text{👤}$$

$$(dbx\text{-ge}) \quad \frac{\mathcal{D}_{x+Ly}^{f(y \rightsquigarrow -c \cdot y)} e \geq c \cdot e}{\{e \bowtie 0\} \{x' = f \ \& \ G\} \{e \bowtie 0\}}, \quad \text{👤}$$

where A is a square matrix, b is a vector, $\infty \in \{>, \geq\}$, x and y are independent variables $y \bowtie x$, and y does not appear in G or f : $y \nmid (G, f)$. In contrast with their usual presentation, our semantic formalisation of the Darboux rules also requires the existence of a third independent variable z such that $z \nmid (G, f)$ because our proof requires two applications of the (dG) rule. More work is needed to provide an alternative proof without these conditions, and to generalise A and b in (dG) to be functions on x that do not mention y . The use of matrices in the (dG) rule was possible due to our previous work on linear systems [19, 34] but further generalisations are possible in terms of bounded linear operators.

Example 11 Below, we provide an alternative (dbx-ge)-based Hoare-style proof that $I \Leftrightarrow (g \geq 0)$ is an invariant for `blood_sugar = loop (ctrl ; dyn)` from Examples 2–4.

$$\begin{aligned}
 & \{I\} \text{ loop (ctrl ; dyn) inv } I \{I\} \\
 & \Leftarrow \{I\} \text{ ctrl ; dyn } \{I\} \\
 & \Leftarrow \{I\} g := g_M \{I\} \wedge \{I\} g' = -g \{I\} \\
 & \Leftarrow (g \geq 0) \leq (g_M \geq 0) \wedge \{g_M \geq 0\} g := g_M \{g \geq 0\} \wedge \mathcal{D}_{g+Ly}^{[g \rightsquigarrow -g, y \rightsquigarrow -1 \cdot y]} g \geq -1 \cdot g \\
 & \Leftarrow \top \wedge \top \wedge -g \geq -g \Leftarrow \top.
 \end{aligned}$$

The second implication follows by (h-seq), the third one by (h-cons) and (dbx-ge), and the last one is true by (h-assign), definitions, and the framed derivative rules. This concludes the proof. As previously noted, differential induction cannot certify that I is an invariant of $g' = -g$. Although its invariance could be verified with the flow as in Example 3, it is not always straightforward to find the solution to a system of ODEs (see Sects. 7 and 8). Hence, in dL-style reasoning, one sometimes needs to embed the ODEs into a higher-dimensional space to prove invariance. Despite extant Isabelle's HOL-based proof strategies to certify this example [49], our formalisation of the Darboux rules expands our pool of methods to tackle similar problems in the style of dL. $\square$

6 Reasoning Components

Here, we describe our recently improved support for proof automation in our verification framework. Specifically, we discuss the proof methods we have developed using Isabelle's Eisbach tool [44] and the underlying formalisations of mathematical concepts required to make the automation effective. That is, our methods not only employ the proof rules introduced in Sects. 3.5 and 5, but we also add lemmas and formalisations in this section that aid in making previously described procedures hidden from the user. Thus, our methods often discharge any side conditions generated in the verification process.

As noted in Example 2, verification problems for hybrid programs often follow the pattern `loop (ctrl ; dyn) inv I`. That is, they are an iteration of a discrete controller intervening in a continuous dynamical system. We can therefore apply invariant reasoning with the laws (h-loopi)() and (h-whilei)(), meaning that often the main task requires verifying $\{I\} \text{ ctrl ; dyn } \{I\}$. As foretold at the end of Sect. 3.5.2, two workflows can be applied to verify this Hoare triple. If the system of ODEs is solvable, then we can insert the flow using law (wlp-flow), and VCG becomes purely equational using the forward box laws (Sect. 3.5.1). This approach is implemented in our proof method `wlp_full` (described below in Sect. 6.4). Alternatively, if a solution is not available, we can find a suitable invariant I and use our differential induction proof method `dInduct`, and its variants (Sect. 6.6), to verify $\{I\} \text{ dyn } \{I\}$. In the remainder, we describe the proof methods and formalisations that support each of these

workflows. Specifically, for `wlp_simp` we describe our automation of the certification of differentiation, Lipschitz continuity, uniqueness of solutions, and the integration of computer algebra systems (CASS) into IsaVODEs. On the `dInduct` side, we provide proof methods for automating differential induction, weakening, and differential ghosts. Finally, we lay the foundations for automating the proof obligations that depend on analysing real-valued functions. We do this by formalising and proving well-known derivative test theorems ubiquitous in calculus.

6.1 Automatic Certification of Differentiation

Our tactic `vderiv` for automatically discharging statements of the form $f' = g$ for functions $f, g : \mathbb{R} \rightarrow \mathcal{C}$ has been described before [50] but it is a component of other tactics and we have extended its capabilities here. Essentially, by the chain rule of differential calculus, derivative laws often require further certifications of simpler derivatives. A recursive procedure emerges where the certification of $f' t = g t$ with $f = f_1 \circ \dots \circ f_n$ is determined by a list of proven derivative rules $(f_i(h t))' = f'_i(h t) \cdot (h' t)$ for $i \in \{1, \dots, n\}$ together with $t' = 1$. The tactic `vderiv` recursively applies these rules until it determines that

$$f' t = (f'_n t) \cdot \prod_{i=1}^{n-1} f'_i((f_{i+1} \circ \dots \circ f_n) t).$$

Then, it uses Isabelle's support for (the undecidable problem of) higher-order unification to try to show that

$$g t = (f'_n t) \cdot \prod_{i=1}^{n-1} f'_i((f_{i+1} \circ \dots \circ f_n) t).$$

For this work, we have added derivative rules for the real-valued exponentiation $\exp(-)$, the square root $\sqrt{-}$, the tangent $\tan(-)$ and cotangent $\cot(-)$ trigonometric functions, and vectors' inner products $*_R$ and norms $\|-\|$. The tactic `vderiv` is an integral part of those described below, and it is tacitly used in our example of Sect. 8.1.

6.2 Automatic Certification of Lipschitz Continuity

As evidenced in Examples 1–4, verification of hybrid systems might depend on knowing that there is a unique solution for a system of differential equations. In practice, certifying the existence and uniqueness of solutions with a general-purpose prover has required finding the Lipschitz-continuity constant [21, 50]. If it exists, then by the Picard–Lindelöf theorem (see Sect. 3.1), there is an interval around the initial time where solutions to the system of ODEs are unique. Alternatively, a domain-specific prover can restrict the specification language to a fragment where uniqueness is guaranteed [59] albeit limiting the space of verifiable dynamics. Here, we provide the foundation for allowing a general-purpose prover to automatically certify the uniqueness of solutions to IVPs. Namely, we formalise the well-known fact that continuously differentiable (C^1) functions are Lipschitz continuous. The statement of the Isabelle lemma is shown below.

```
lemma c1_local_lipschitz:
  fixes f::"real  $\Rightarrow$  ('a::{banach,perfect_space})  $\Rightarrow$  'a"
  assumes "open S" and "open T"
  and c1hyp: " $\forall \tau \in T. \forall s \in S. D(f \tau) \mapsto \mathcal{D} \text{ (at } s \text{ within } S)$ "
```



```

and "continuous_on S  $\mathcal{D}$ "
shows "local_lipschitz T S f"
(proof)

```

Then, a procedure emerges when applying ([wlp-flow](#)) or ([fdia-flow](#)):

1. Users try to rewrite a correctness specification that includes an evolution command. That is they try to rewrite $|(x' = f \& G)_{\mathcal{U}}^{t_0}] Q s$ or $|(x' = f \& G)_{\mathcal{U}}^{t_0}] Q s$.
2. In order to guarantee that ([wlp-flow](#)) or ([fdia-flow](#)) are applicable, users need to show that there is a flow φ for the ODEs $x' = f$. This is formalised in Isabelle/HOL as the proof obligation `local_flow_on f x T S  $\varphi$` .
3. Unfolding this predicate's definition yields the obligation `local_lipschitz T S f` to show that the vector field f is Lipschitz continuous on T .
4. Then, users can apply our lemma `c1_local_lipschitz` above which requires them to show that (1) T and S are open sets and that (2) the derivative of $f \tau$ is some function $\mathcal{D}$ that (3) is continuous on S . Users can either supply $\mathcal{D}$ or let Isabelle try to reconstruct it through its automation later in the proof.
5. Users discharge the remaining proof obligations: openness, continuity, differentiability, and original obligation with $|(x' = f \& G)_{\mathcal{U}}^{t_0}] Q s$ rewritten by ([wlp-flow](#)).

Example 12 We illustrate the procedure for certifying the uniqueness of solutions in Isabelle below by formalising part of the argument of Example 3. Recall the definitions of the control and dynamics of the problem.

abbreviation "ctrl $\equiv$ IF $g \leq g_m$ THEN $g ::= g_M$ ELSE skip"

abbreviation "dyn $\equiv$ { $g' = -g$ }"

abbreviation "flow $\tau \equiv$ [$g \rightsquigarrow g * \exp(-\tau)$]"

abbreviation "blood_sugar $\equiv$ LOOP (ctrl; dyn) INV ($g \geq 0$)"

The procedure appears in a partial apply-style Isabelle proof below. We label each procedure step with its number and add Isabelle outputs after each **apply** command.

```

lemma "{g  $\geq 0$ } blood_sugar {g  $\geq 0$ }"
  apply (wlp_simp) — the forward box  $|g' = -g| 0 \leq g$  appears
  apply (subst fbox_solve[ where  $\varphi = \text{flow}$ ]) — Step (1)
  Isabelle's output: local_flow_on [g  $\rightsquigarrow$  g] g  $\mathbb{R}$   $\mathbb{R}$  flow — Step (2)
  apply ((clarsimp simp: local_flow_on_def?, unfold_locales; clarsimp?))
  Isabelle's output: local_lipschitz  $\mathbb{R}$   $\mathbb{R}$  ( $\lambda t. [g \rightsquigarrow g] \downarrow_{\text{subst } g}$ ) — Step (3)
  apply (rule c1_local_lipschitz) — Step (4)
  Isabelle's outputs: open  $\mathbb{R}$ , open  $\mathbb{R}$ , continuous_on  $\mathbb{R}$   $\mathcal{D}, \dots$ 
  (proof) — Step (5)

```

In practice, certifying openness of $\mathbb{R}$ or intervals $(a, b) = \{x \mid a < x < b\}$ with $a \leq b$ is automatic thanks to Isabelle's simplifier. Finding derivatives or checking continuity is not always as straightforward but simple linear combinations are automatically certifiable. We have bundled the procedure for certifying the uniqueness of solutions to IVPs in an Isabelle tactic `local_flow_on_auto` described in Sect. 6.3 but below we focus on automating the certification of Lipschitz continuity. $\square$

In Example 1, we use the fact that continuously differentiable (C^1) functions are Lipschitz continuous to argue that a system of ODEs f has unique solutions by the Picard–Lindelöf theorem. Thus, formalisation of the rule `c1_local_lipschitz` is the first step towards automating

the certification of the uniqueness of solutions to IVPs and a crucial step in hybrid system verification (via flows) in general-purpose proof assistants. Next, we provide tactics `c1_lipschitz` and `c1_lipschitzI` to make certification of Lipschitz continuity as seamless in VCG as in pen and paper proofs, like that of Example 1. Our tactics automate an initial application, in a backward reasoning style, of our lemma `c1_local_lipschitz`. The tactics discharge the emerging proof obligations by replicating the behaviour of `vderiv` but using rules for continuity and Fréchet differentiability available in Isabelle's HOL-Analysis library. The difference between both tactics is that `c1_lipschitzI` allows users more control by enabling them to supply the derivative $\mathcal{D}$. We exemplify `c1_lipschitz` in VCG below:

```
lemma "vwb_lens x  $\implies$  vwb_lens y  $\implies$  vwb_lens z
 $\implies$  x  $\bowtie$  y  $\implies$  x  $\bowtie$  z  $\implies$  y  $\bowtie$  z
 $\implies$  local_lipschitz UNIV UNIV ( $\lambda t::\text{real}.$  [x  $\rightsquigarrow$  $y, y  $\rightsquigarrow$  $z]  $\downarrow_{\text{Subst } x +_L y \text{ } s}$ )"
by c1_lipschitz

lemma "vwb_lens x  $\implies$  local_lipschitz UNIV UNIV ( $\lambda t.$  [x  $\rightsquigarrow$  1 - $x]  $\downarrow_{\text{Subst } x \text{ } s}$ )"
by c1_lipschitz

lemma "vwb_lens x  $\implies$  x  $\bowtie$  y
 $\implies$  local_lipschitz UNIV UNIV ( $\lambda t::\text{real}.$  [x  $\rightsquigarrow$  - ($y * $x)]  $\downarrow_{\text{Subst } x \text{ } s}$ )"
by c1_lipschitz
```

In applications, we use our tactics on framed substitutions, e.g. $[x \rightsquigarrow 1 - \$x] \downarrow_{\text{Subst } x}$, that represent systems of ODEs f . VCG also requires assumptions of lens-independence and satisfaction of lens laws due to our use of shallow expressions and frames. These are automatically provided by our `dataspace` Isabelle command and taken into account in our `c1_lipschitz` tactics. The tactic `c1_lipschitz` is indirectly used in the verification problem in Sect. 8.1. Both tactics automatically discharge proof obligations where the ODEs' (the vector field f) form a linear system of ODEs. This already yields polynomial and transcendental functions as solutions φ to these systems. Accordingly, users can employ our tool support for linear systems of ODEs [34]. We leave the automation of Lipschitz continuity certification for non-linear systems as future work. Alternative proof strategies like differential induction (see Sect. 8) are also available for cases when these tactics fail.

6.3 Automatic Certification of the Flow

In Sect. 6.2, we describe a procedure to verify the partial correctness of an evolution command by supplying the flow to its system of ODEs. Here, we automate the flow certification part of this procedure in a tactic `local_flow_on_auto`. This proof method calls our two previously described tactics `c1_lipschitz` and `vderiv`. The first one discharges the Lipschitz continuity requirement from the Picard–Lindelöf theorem to guarantee uniqueness of solutions to the associated IVP. The tactic `vderiv` certifies that the supplied flow φ is a solution to the system of ODEs. The following lines exemplify our tactic usage in hybrid systems verification tasks. Our tactic is robust as it automatically discharges frequently occurring proof obligations. Notice that for each Isabelle lemma below, there is a corresponding (local) Lipschitz continuity example from Sect. 6.2.

```
lemma "vwb_lens x  $\implies$  vwb_lens y  $\implies$  vwb_lens z
 $\implies$  x  $\bowtie$  y  $\implies$  x  $\bowtie$  z  $\implies$  y  $\bowtie$  z
 $\implies$  local_flow_on [x  $\rightsquigarrow$  $y, y  $\rightsquigarrow$  $z] (x +L y) UNIV UNIV
( $\lambda t.$  [x  $\rightsquigarrow$  $z * t2 / 2 + $y * t + $x, y  $\rightsquigarrow$  $z * t + $y])"
by local_flow_on_auto
```

```

lemma "vwb_lens x  $\implies$  local_flow_on [x  $\rightsquigarrow$  - $x + 1] x UNIV UNIV
( $\lambda$ t. [x  $\rightsquigarrow$  1 - exp (- t) + $x * exp (- t)])"
by local_flow_on_auto

lemma "vwb_lens (x::real  $\implies$  's)  $\implies$  x  $\bowtie$  y
 $\implies$  local_flow_on [x  $\rightsquigarrow$  - $y * $x] x UNIV UNIV
( $\lambda$ t. [x  $\rightsquigarrow$  $x * exp (- t * $y)])"
by local_flow_on_auto

```

Crucially, `local_flow_on_auto` certifies the exponential and trigonometric solutions required for these ODEs as evidenced by its successful application on the examples above and in the verification problem of Sect. 8.1.

6.4 Automatic VCG with Flows

In addition to our derivative, Lipschitz continuity, and flow automatic certifications, we have added various tactics for verification condition generation. The simplest one is `wlp_simp`. It merely calls Isabelle's simplifier adding Sect. 3's *wlp* equational laws as rewriting rules except those for finite iterations and while loops which it initially tries to remove with invariant reasoning via (`h-loopi`) and (`h-whilei`). The tactic assumes that the hybrid program has the standard shape `loop (ctrl; dyn) inv I` iterating a discrete controller intervening in a continuous dynamical system as seen in Example 2. From this initial proof method, we create two tactics for supplying flows: the tactic `wlp_flow` takes as input a previously proven flow-certification theorem asserting our predicate `local_flow_on`. Alternatively, users can try to make this certification automatic with the tactic `wlp_solve` which requires as input a candidate solution φ . It calls `local_flow_on_auto` after applying `wlp_simp` and trying (`wlp-flow`) with the input φ . The intention is that both `wlp_flow` and `wlp_solve` leave only proof obligations that require reasoning of first-order logic of real numbers. Complementary tactics `wlp_full` and `wlp_expr_solve` try to discharge these remaining proof obligations automatically leaving raw Isabelle terms in the proof obligations without our syntax translations. We have also supplied simple tactics for algebraic reasoning more interactively. Specifically, we have provided a tactic for distributing factors over additions and a tactic for simplifying powers in multi-variable monomials. See Sect. 8.1 for the application of these in a verification problem.

6.5 Solutions from Computer Algebra Systems

We have integrated two Computer Algebra Systems (CASs), namely SageMath and the Wolfram Engine, into Isabelle/HOL to supply symbolic solutions to ODEs. The user can make use of the integration via the `find_local_flow` command, which supplies a solution to the first ODE it finds within the current subgoal.

Below, we show an application of our integration and its corresponding output in Isabelle. Users can click the greyed-out area to automatically insert the suggestion.

```

lemma local_flow_example:
  "{x > 0} x ::= 1; {x' = 1} {x > 1}"
find_local_flow
Output:
  Calling Wolfram...
   $\lambda$ t. [x  $\rightsquigarrow$  t + $x]
  try this: apply (wlp_solve " $\lambda$ t. [x  $\rightsquigarrow$  t + $x]")

```

Here, our plugin requests a solution to the ODE $x' = 1$ from the Wolfram Engine. A λ -abstraction denotes the solution, which inputs the current time and has in its body the constructed substitution (see [lens-subs](#)). It provides the value for each continuous variable at time t . In this case, the substitution $[x \rightsquigarrow t + x]$ represents the solution $x\ t = t + x\ 0$.

Our integration performs the following steps in order to produce a solution:

1. Retrieve an Isabelle term describing the system of ODEs.
2. Convert the term into an intermediate representation.
3. Use one of the CAS backends to solve the ODE.
4. Convert the solution to an Isabelle term.
5. Certify the solution using the `wlp_solve` Isabelle tactic.

We consider each of these stages below.

Intermediate representation

To have an extensible and modular interface, we implement (1) an intermediate data structure for representing ODEs and their solutions, and (2) procedures for translating back and forth between Isabelle and our intermediate representation. This allows us to capture the structure of the ODEs without the added overhead of Isabelle syntax. It also gives us a unified interface between Isabelle and any CAS.

Our plugin assumes that a system of ODEs is expressed as a substitution of the form $[x \rightsquigarrow e, y \rightsquigarrow f, \dots]$, as described in Sect. 5.2. Each expression $(e, f, \dots)$ gives the derivative expression for each corresponding variable, potentially in terms of other variables. Naturally, for an ODE, these expressions are formed using only arithmetic operators and mathematical functions. We therefore derive the following constrained grammar for arithmetic expressions:

```
datatype AExp = NConst of string | UOp of string * AExp |
              BOp of string * AExp * AExp | NNat of int | NInt of int |
              NReal of real | CVar of string | SVar of string | IVar
```

`NConst` represents numeric constants, such as e or π . `UOp` is for unary operations and `BOp` is for binary operations. Both of these take a name, and the number of required parameter expressions. The name corresponds to the internal name for the operator in Isabelle/HOL. The operator “+”, for example, has the name `Groups.plus_class.plus`. We then have three constructors for numeric constants: `NNat` for naturals, `NInt` for integers and `NReal` for reals. We have three characterisations of variables: `CVar` are arbitrary but fixed variables, `SVar` are mutable (state) variables, and `IVar` represents the independent variable of our system, usually time.

A system of ODEs is then modelled simply as a finite map from variable names to derivative expressions. Converting back and forth between Isabelle terms (i.e. elements of the `term` datatype in Isabelle/ML) and the `AExp` datatype consists of recursing through the structure of the term, using a lookup table to translate each named arithmetic function.

Integrating a CAS into IsaVODEs consists of three separate components: a translation from the intermediate representation to the CAS input format, an interface with the CAS to obtain a solution, and a translation from the solution back into the required format.

Wolfram Engine The Wolfram Engine is the CAS behind WolframAlpha and Mathematica [76]. It is one of the leading CASs on the market, with powerful features for solving differential equations, amongst many other applications. The Wolfram language provides the `DSolve` function which produces solutions to various kinds of differential equations.

The basic building block for this integration is a representation of Wolfram expressions as an ML datatype. In the Wolfram language, everything is an expression [75], so this is sufficient for a complete interface. We represent these expressions as follows:

```
datatype expr = Int of int
                | Real of real
                | Id of string
                | Fun of string * expr list
                | CurryFun of string * expr list list
```

Real and Int represent real and integer numbers respectively. Id represented an identifiers, and Fun represents functions with only one list of arguments, which are distinguished from curried functions CurryFun with multiple sets of arguments.

Our approach for translation from AExp to expr is the following:

1. Generate an alphabetically ordered variable mapping to avoid name clashes and ease solution reconstruction.
2. Traverse the expression tree and translate each term to Wolfram.
3. Wrap in the Wolfram DSolve function, supplying the state and independent variables along with our ODE.

Once we have a well-formed Wolfram expression, we use the wolframscript command-line interface to obtain a solution in the form of a list of Wolfram Rule expressions, which are isomorphic to our substitutions. We can parse the result back into the expr type, and translate this into the AExp type using a translation table for function names and the variable name mapping we constructed.

SageMath SageMath [72] is an open-source competitor to the Wolfram Engine. It is accessed via calls to a Python API. It integrates several open-source CASs to provide its functionality, choosing the best implementation for a particular symbolic computation. This makes SageMath an ideal target for integration with Isabelle.

The translation process is similar to our Wolfram Engine integration, but we also apply some preprocessing to the ODEs to make the solving more efficient. Since the CASs integrated with SageMath are better at solving smaller ODEs, we can improve their performance by rewriting the following kinds of ODEs:

1. Systems of ODEs from a higher order ODE (where an extra variable has been introduced to make all derivatives first order) can be recast into their higher-order form. For example, the ODE $x' = 2x + y$, $y' = x$ can be rewritten as $x'' = 2x + x'$.
2. Independent equations that can be solved independently by the CAS. For example, in the system $x' = x$, $t' = 1$, the variables x and t can be solved independently.

Capabilities There are two main cases where the integration fails to produce an Isabelle certification for a solution, despite the CAS returning a solution.

The first case happens when the provided solution contains a function not implemented in Isabelle. Instances of this case include functions defined in terms of integrals such as the error function or the Bessel function.

The second case concerns solutions which are undefined somewhere within their domain. Examples of this case include divisions by 0, which users can address by manually specifying the domain of the solution in Isabelle.

Despite these shortcomings, the CAS integration proves very effective at leveraging the solutions provided by CASs, as a large class of ODEs can be automatically solved and certified with minimal input from the user.

6.6 Automatic Differential Invariants

We have developed three main Eisbach proof methods for automating differential invariant proofs: `dWeaken`, `dInduct`, and `dGhost`, which implement differential weakening, induction, and ghosts, respectively [50, 61]. We show the Eisbach declaration for some of them below. The first method, `dWeaken`, applies the differential weakening law (theorem `diff_weak_on`) [59] to prove $\{P\} \{x' = f \mid G\} \{Q\}$, when $G \Rightarrow Q$. Proof of the implication is attempted via the `auto` proof method.

```
method dWeaken = (rule_tac diff_weak_on_rule; expr_simp;
  auto simp add: field_simps)
```

Application of differential induction (see Sect. 5.3) is automated by the `dInduct` proof method. Conjunctions and disjunctions are split via `(h-conji)` and `(h-disji)` (in Isabelle: `hoare_invs`). To prove atomic goals $\{I\} \{x' = f \mid G\} \{I\}$, our method (1) applies rules `(dinv-eq)-(dinv-less)` to produce a framed derivative expression (in Isabelle: `lderv_rules`), and (2) calculates derivatives using the laws (1)-(7) (i.e. `framed_derivs` below), substitution laws, and basic simplification laws. This yields derivative-free equality or inequality predicates. The `dInduct` method uses only the simplifier for calculating invariants, and so it is both efficient and yields readable VCs.

```
method dInduct = ((intro hoare_invs)?; subst fbox_diff_inv_on;
  rule_tac lderv_rules; simp add: framed_derivs ldifferentiable closure
  usubst unrest_ssubst unrest usubst_eval)
```

For cases requiring deduction to solve the VCs, we have implemented `dInduct_auto`, which applies `expr-auto` after `dInduct`, plus further simplification lemmas from HOL-Analysis. Ultimately such heuristics should be based on decision procedures [10, 17, 33, 42], as oracles or as verified components. 

Should `dInduct_auto` fail, our `diff_inv_on` methods use the vector derivative laws from Isabelle's ODEs library [35, 50]. These are more interactive methods as users need to provide the derivatives of both sides of the (in)equality. Yet, this provides more fine-grained control and makes the tactics more likely to succeed. 

While `dInduct_auto` suffices for simpler examples, differential induction must often be combined with weakening and cut rules. These rules have been explained elsewhere [50, 61]. This process is automated by a search-based proof method called `dInduct_mega`. The following steps are executed iteratively until all goals are proved or no rule applies: (1) try any fact labelled with attribute `facts`, (2) try differential weakening to prove the goal, (3) try differential cut [58] to split it into two differential invariants, (4) try `dInduct_auto`. The rules are applied using backtracking so that if one rule fails, another one is tried.

```
method dInduct_mega uses facts =
  (fact facts — (1) Try any facts we have provided
  | (dWeaken ; force) — (2) Try differential weakening
  | rule_tac diff_cut_on_split'
  | rule_tac diff_cut_on_split — (3) Try differential cut (two options)
  | rule_tac hoare_if_then_else_inv
  | (dInduct_auto) — (4) Try differential induction
  )+
```

The method `dInduct_mega` is called by `dInv`, which we applied in Sect. 2. The `dInv` method allows us to prove a goal of the form $\{P\} \{x' = f \mid G\} \{Q\}$ by supplying an assertion I ,

and proving that this is an invariant of $\{x' = f \mid G\}$ and also that $P \Rightarrow I$. This method also requires us to apply differential cut and weakening laws.

Finally, the differential ghost law (dG) is automated through the dGhost proof method. We have complemented these tactics with sound rules from differential dynamic logic. Together with the tactics described in this Section, they enable users to seamlessly prove properties about hybrid systems in the style of dL. Yet, users of the general-purpose prover can also use the interactive style provided by Isabelle's Isar scripting language [49]. See Sect. 8.2 for an example where our tactics largely automate a proof of invariance.

6.7 Derivative Tests

To culminate this section, we lay the foundations for further automating the verification process in our framework. Both wlp_full and dInduct methods cannot certify complex real arithmetic proof obligations after discharging any derivative-related intermediate steps. In many cases, this requires determining the local minima and maxima of expressions and using them for further argumentation. A well-known application of differentiation is the analysis of real-valued functions: determining their local minima and maxima or where they are increasing or decreasing. Optimisation problems in physics and engineering frequently require this kind of analysis. Behind it, there are two key results generally called the *first* and *second derivative tests*. We formalise and prove these basic theorems in Isabelle/HOL, setting the foundation for automatic certification of the analysis of real-valued functions (see Sect. 8.3). We start by defining increasing/decreasing functions and local extrema:

definition "increasing_on T f $\longleftrightarrow$ ($\forall x \in T. \forall y \in T. x \leq y \longrightarrow f\ x \leq f\ y$)"

definition "decreasing_on T f $\longleftrightarrow$ ($\forall x \in T. \forall y \in T. x \leq y \longrightarrow f\ y \leq f\ x$)"

definition "strict_increasing_on T f $\longleftrightarrow$ ($\forall x \in T. \forall y \in T. x < y \longrightarrow f\ x < f\ y$)"

definition "strict_decreasing_on T f $\longleftrightarrow$ ($\forall x \in T. \forall y \in T. x < y \longrightarrow f\ y < f\ x$)"

definition "local_maximum_at T f x $\longleftrightarrow$ ($\forall y \in T. f\ y \leq f\ x$)"

definition "local_minimum_at T f x $\longleftrightarrow$ ($\forall y \in T. f\ y \geq f\ x$)"

We also prove simple consequences of these definitions on closed intervals $[a, b]$ denoted in Isabelle as $\{a..b\}$. For instance, we prove the transitivity of the increasing and decreasing properties over consecutive intervals and their relationship to local extrema. We exemplify some of these results below and refer to our repository for all proved properties. 

lemma increasing_on_trans:

```
fixes f :: "'a::linorder  $\Rightarrow$  'b::preorder"
assumes "a  $\leq$  b" and "b  $\leq$  c"
and "increasing_on {a..b} f" and "increasing_on {b..c} f"
shows "increasing_on {a..c} f"
unfolding increasing_on_def
by (auto simp: Ball_def) (metis dual_order.trans nle_le)
```

lemma increasing_on_local_maximum:

```
fixes f :: "'a::preorder  $\Rightarrow$  'b::preorder"
assumes "a  $\leq$  b" and "increasing_on {a..b} f"
shows "local_maximum_at {a..b} f b"
by (auto simp: increasing_on_def local_maximum_at_def)
```

lemma incr_decr_local_maximum:

```
fixes f :: "'a::linorder  $\Rightarrow$  'b::preorder"
```

```

assumes "a ≤ b" and "b ≤ c"
and "increasing_on {a..b} f" and "decreasing_on {b..c} f"
shows "local_maximum_at {a..c} f b"
unfolding increasing_on_def decreasing_on_def local_maximum_at_def
by auto (metis intervalE linorder_le_cases)

```

Crucially, the proofs of all these results only require unfolding definitions and calling Isabelle's *auto* proof method and *Sledgehammer* tool [53] to discharge the last proof obligation. We then use our definitions of increasing and decreasing functions to state and prove the first derivative test. It states that if the derivative of a function is greater (resp. less) than 0 over an interval T , then the function is increasing (resp. decreasing) on that interval. The simple 2-line proof above uses an intermediate lemma `has_vderiv_mono_test` hiding our full proof of this basic result. 

```

lemma first_derivative_test:
  assumes T_hyp: "is_interval T"
  and d_hyp: "D f = f' on T"
  shows "∀x∈T. (0::real) ≤ f' x ⇒ increasing_on T f"
  and "∀x∈T. f' x ≤ 0 ⇒ decreasing_on T f"
  unfolding increasing_on_def decreasing_on_def
  using has_vderiv_mono_test[OF assms] by blast

```

For the second derivative test, we formalise a frequently used property of continuous real-valued functions. Namely, that if a function maps a point t to some value above (resp. below) a threshold c , then there is an open set around t filled with points mapped to values above (resp. below) the threshold. This result and similar ones in terms of open balls with fixed radius around t appear in our formalisations. For these, we also complement Isabelle's library of topological concepts with the definition of *neighbourhood* and some of its properties and characterisations. 

```

definition "neighbourhood N x ⟷ (∃X. open X ∧ x ∈ X ∧ X ⊆ N)"

lemma continuous_on_Ex_open_less:
  fixes f :: "'a :: topological_space ⇒ real"
  assumes "continuous_on T f"
  and "neighbourhood T t"
  shows "f t > c ⇒ ∃X. open X ∧ t ∈ X ∧ X ⊆ T ∧ (∀τ∈X. f τ > c)"
  and "f t < c ⇒ ∃X. open X ∧ t ∈ X ∧ X ⊆ T ∧ (∀τ∈X. f τ < c)"

```

Finally, the second derivative test states that if the derivative of a real-valued function is 0 at a point t , and its second derivative is continuous and positive (resp. negative) at t , then the original function has a local minimum (resp. maximum) at t .

```

lemma second_derivative_test:
  assumes "continuous_on T f'"
  and "neighbourhood T t"
  and f': "D f = f' on T"
  and f'': "D f' = f'' on T"
  and "f' t = (0 :: real)"
  shows "f'' t < 0
    ⇒ ∃a b. a < t ∧ t < b ∧ {a..b} ⊆ T ∧ local_maximum_at {a..b} f t"
  and "f'' t > 0
    ⇒ ∃a b. a < t ∧ t < b ∧ {a..b} ⊆ T ∧ local_minimum_at {a..b} f t"
  unfolding local_maximum_at_def local_minimum_at_def
  using has_vderiv_max_test[OF assms] has_vderiv_min_test[OF assms]
  by blast+

```

As before, we provide the complete argument in the proof of our lemmas `has_vderiv_max_test` and `has_vderiv_min_test`. We refer interested readers to our repository for complete results.

 Sect. 8 provides an example where we use these derivative tests to reason about real-arithmetic properties and establish the progress of a dynamical system. Beyond that, this subsection showcases the openness of our framework. Anyone can formalise well-known analysis concepts that provide background theory engineering to increase proof automation or that generalise extant verification methods.

We have presented various tactics that increase proof automation for hybrid systems verification in our framework. The automation supports both, solution and invariant-based reasoning. The definition of the tactics and their testing covers more than 500 lines of Isabelle code. From these 500 lines, the tactics setup (definitions and lemmas) comprises approximately 200 lines. This number does not take into account our described formalisations. The tactics have simplified our verification experience by discharging certifications of the uniqueness of solutions or differential inductions. We have used them extensively in a set of 66 hybrid systems verification problems. See Sect. 7 for more details.

7 Evaluation

For the evaluation of our verification framework, we have tackled 66 problems of the Hybrid Systems Theorem Proving (HSTP) category from the 9th International Workshop on Applied Verification of Continuous and Hybrid Systems (ARCH22) [47] Friendly Competition. We provide several classifications for them below:

- *Regular, continuous or hybrid*: Of the 66 problems, 5 use regular programs, 32 are verifications of continuous dynamics and 29 are verifications of hybrid programs.
- *Purpose*: The first 9 problems test the tool's ability to handle the interactions between hybrid programs' constructors through various orders of loops, tests, assignments and dynamics. The next 30 problems test the tool's ability to tackle different kinds of continuous dynamics: one evolution command after another, an evolution command with many variables at once, and dynamics that in $d\mathcal{L}$ require invariants, ghosts, differential cuts, weakenings, or Darboux rules. 21 problems come from tutorials [57, 62] (9 and 12 respectively) on how to model and prove hybrid systems in $d\mathcal{L}$. They include event and time-triggered controls for straight-line motion and two-dimensional curved motion. 3 hybrid programs come from a case study on the verification of the European train control system protocol [54]. The remaining 3 involve linear and nonlinear dynamics.
- *Competition's categories*: Relative to the competition, we took 61 (all) examples from the Design Shapes category that test the tools' verification features. We took 2 problems from the nonlinear continuous models collected from the literature on continuous safety verification and invariant generation for nonlinear systems, and we took 3 problems from the case study benchmarks that test the tools' scalability and efficiency on examples of significant size.
- *Dynamics*: 40 problems have linear dynamics, 21 have nonlinear dynamics, and the remaining 5 are regular programs (without ODEs). The average of continuous variables per problem is 1.9 out of 3.8 with a maximum of 6 continuous variables and 13 variables in a problem. See our Appendix A for more details. 

We fully proved in Isabelle 58 of the 66 problems. Of the remaining 8, we proved 4 with our verification framework while leaving some arithmetic certifications to external computer

algebra systems. We could not find the proofs for 2 of the remaining 4 problems while the last 2 require the generalisations of the Ghost rule discussed in Sect. 5.4 or a different proof method [49]. Restricting the comparison to those benchmark problems in the Design Shapes category of the competition [47] in the proof-interaction/scripted format, KeYmaera X reports solving 60 of the 61 problem examples. Their only unsolved problem requires a witness for an existential quantifier that no team could supply. Similarly, the HHLPy prover reports 50 of the 61 design shapes problems successfully solved. Their main issue was the difficulty of translating problems from a $d\mathcal{L}$ specification to an HHLPy specification. Finally, we solved 59 of 61 of the problems in that category, with 55 of them fully proved without involving a CAS. For an in-depth comparison, we refer readers to the competition's full report [47]. Thus, despite its recent implementation, the performance of IsaVODEs is comparable to state-of-the-art tools thanks to the advantages of using a well-established prover like Isabelle to build verification tools.

In general, our first approach to solving all 66 problems was a combination of our *wlp*'s tactics with a supplied solution. Yet, we were only able to solve 39 of them with this approach. The remaining 19 problems required us to employ higher-order logic (HOL) methods or $d\mathcal{L}$ techniques like differential induction, ghosts, cuts, weakenings or Darboux rules. In particular, we had to use Isabelle's HOL and analysis methods to tackle three nonlinear dynamics problems which immediately require us to increase our interaction with the tool. A generalisation of our invariance certification methods (Sect. 5.4) would alleviate this [49] because as solutions to the systems of ODEs become more complex, their certification is more difficult. This explains why invariant reasoning becomes prominent in the remaining 16 solved problems. Readers can see our Appendix A for tables summarising the performance of IsaVODEs per problem.

For 14 problems, we provided several proofs to exemplify our tool's diversity of methods. Between providing solutions and using differential induction, neither method is comparatively easier to use than the other in the 14 problems tested. Most of the time they end up with the same number of lines of code (LOC) per proof and whenever one has more LOC for a problem, a different problem favours equally the other method. Quantitatively speaking, we used the solution method 37 times and the induction method 26 times. We used differential ghosts 4 times and our Darboux rule twice. In terms of LOC, the average length of the statement of a problem is 3.72 lines with a median and mode of 1 totalling 246 LOCs for all problem statements. The average number of LOC of the problems' shortest proofs is 8.11 with a median of 3 and a mode of 1 totalling 511 LOC for problems' proofs. The shortest proof for a problem is 1 while the longest is 98 LOC. These figures do not take into account any additional lemmata about real numbers necessary for having fully certified proofs in Isabelle/HOL. In general, 23 of the 62 (at least partially) solved problems require the assertion of real arithmetical facts for their full verification. Thus, automation of first-order logic arithmetic for real numbers in general-purpose ITPs is highly desired for the scalability of end-to-end verification within them. Otherwise, trust in computer algebra tools will be necessary.

Our additions of tactics to the verification framework have highly reduced the number of LOC to verify a problem as evidenced by the fact that 48 benchmark problems are solved with a single call to one of our tactics. Our addition of the fact that C^1 -functions are Lipschitz continuous has largely contributed to the success of this automation: 18 of our proofs call our tactic `wlp_solve` and 9 use `local_flow_on_auto`. Finally, thanks to our shallow expressions and our nondeterministic assignments, our formalisation of the benchmark problems is now fully faithful to the ARCH22 competition-required $d\mathcal{L}$ syntax.

8 Examples

In this section, we showcase the benefits of our contributions by applying them in some of the ARCH22 competition benchmarks and examples of our own.

8.1 Rotational Dynamics 3

Our first problem illustrates the integration of all our features for ODEs. It describes the preservation of $I \Leftrightarrow d_1^2 + d_2^2 = w^2 \cdot p^2 \wedge d_1 = -w \cdot x_2 \wedge d_2 = w \cdot x_1$ for the ODEs $x_1' = d_1, x_2' = d_2, d_1' = -w \cdot d_2, d_2' = w \cdot d_1$. Observe that the relationship between d_1 and d_2 in the system of ODEs is similar to that of scaled sine and cosine functions. Moreover, the invariant states a Pythagorean relation among them. Thus, we can expect the flow to involve trigonometric functions and the problem to be solved with differential invariants in $d\mathcal{L}$ due to its previously limited capabilities to explicitly state these functions [29]. Indeed, differential induction in Isabelle/HOL can prove this example in one line:

```
lemma "(d12 + d22 = w2 * p2 ∧ d1 = - w * x2 ∧ d2 = w * x1)e ≤
  |{x1' = d1, x2' = d2, d1' = - w * d2, d2' = w * d1}|
  (d12 + d22 = w2 * p2 ∧ d1 = - w * x2 ∧ d2 = w * x1)"
by (intro fbox_invs; diff_inv_on_eq)
```

In the proof above, the application of the lemma `fbox_invs` as an introduction rule splits the Hoare-triple in three invariant statements, one for each conjunct. The semicolon indicates to Isabelle that the subsequent tactic should be applied to all emerging proof obligations. Therefore, our tactic `diff_inv_on_eq` implementing differential induction for equalities (`dinv-eq`) is applied to each of the emerging invariant statements, which concludes the proof.

Nevertheless, we can also tackle this problem directly via the flow. Despite the fact that we suspect that the solutions involve trigonometric functions, obtaining the general solution is time-consuming. Therefore, we call our integration between the Wolfram language and Isabelle/HOL `find_local_flow` to provide the solution for us. We can use the supplied solution in the proof.

```
lemma "w ≠ 0 ⇒ (d12 + d22 = w2 * p2 ∧ d1 = - w * x2 ∧ d2 = w * x1)e ≤
  |{x1' = d1, x2' = d2, d1' = - w * d2, d2' = w * d1}|
  (d12 + d22 = w2 * p2 ∧ d1 = - w * x2 ∧ d2 = w * x1)"
find_local_flow
apply (wlp_solve "λt. [d1 ~> $d1 * cos (t * w) + - 1 * $d2 * sin (t * w),
  d2 ~> $d2 * cos (t * w) + $d1 * sin (t * w),
  x1 ~> $x1 + 1/w * $d2 * (- 1 + cos (t * w)) + 1/w * $d1 * sin (t * w),
  x2 ~> $x2 + 1/w * $d1 * (1 + - cos (t * w)) + 1/w * $d2 * sin (t * w)]")
apply (expr_auto add: le_fun_def field_simps)
subgoal for s t
  apply mon_pow_simp
  apply (mon_simp_vars "get_x1 s" "get_x2 s")
  using rotational_dynamics3_arith by force
done
```

The first line of the proof applies our tactic `wlp_solve` with the suggested solution from `find_local_flow`. The solution's syntax specifies one expression per variable in the system of ODEs. As explained in Sect. 6, the tactic `wlp_solve` applies the rule (`wlp-flow`), certifies that the corresponding vector field is Lipschitz-continuous by calling our tactic `c1_lipschitz`, and certifies that it is indeed the solution to the system of ODEs via our tactic `vderiv`. The `subgoal`

command allows us to specify the name of the variables in the proof obligation so that we can use those names in our subsequent tactics. Our new tactic `mon_pow_simp` simplifies powers in the monomial expressions of the proof obligation. In the next line, our tactic `mon_simp_vars` calls `mon_pow_simp` twice but reorders factors in the order of its inputs `get_x1 s` and `get_x2 s`. The last line supplies the lemma `rotational_dynamics3_arith` (below) whose proof was provided by Sledgehammer [53].

```
lemma rotational_dynamics3_arith:
  "w^2 * (get_x1 s)^2 + w^2 * (get_x2 s)^2 = p^2 * w^2
  ⇒ w^2 * ((cos (t * w))^2 * (get_x1 s)^2)
    + (w^2 * ((sin (t * w))^2 * (get_x1 s)^2)
    + (w^2 * ((cos (t * w))^2 * (get_x2 s)^2)
    + w^2 * ((sin (t * w))^2 * (get_x2 s)^2))) = p^2 * w^2"
  (proof)
```

This example shows the versatility of using general-purpose proof assistants for hybrid systems verification. Users can provide automation methods for invariant or flow certification. This allows the integration of unverified tools into the verification process. Fast certification enables ITPs to quickly validate CAS' inputs. In terms of our contributions, this example showcases our shallow embedding's intuitive syntax, our integration of the Wolfram language for suggesting simple solutions, and our tactics automating VCG, C1-lipschitz continuity, derivatives certification, and real-arithmetic reasoning.

8.2 Dynamics: Conserved Quantity

We prove that the inequality $I \Leftrightarrow x_1^4 \cdot x_2^2 + x_1^2 \cdot x_2^4 - 3 \cdot x_1^2 \cdot x_2^2 + 1 \leq c$ is an invariant of the system

$$f = \begin{cases} x_1' = 2 \cdot x_1^4 \cdot x_2 + 4 \cdot x_1^2 \cdot x_2^3 - 6 \cdot x_1^2 \cdot x_2, \\ x_2' = -4 \cdot x_1^3 \cdot x_2^2 - 2 \cdot x_1 \cdot x_2^4 + 6 \cdot x_1 \cdot x_2^2, \end{cases}$$

where we abuse notation and "equate" the vector field with its representation as a system of ODEs. In contrast with the previous benchmark, the solution to this system of ODEs is not easy to describe analytically. The following is a subexpression to the solution for x_2 according to Wolfram|Alpha

$$\left(\int_1^t \frac{1}{\sqrt{\tau^2(\tau^6 - 6\tau^4 + 9\tau^2 + c_1)} \sqrt{\frac{-\tau^4 + 3\tau^2 - \sqrt{\tau^2(\tau^6 - 6\tau^4 + 9\tau^2 + c_1)}}{\tau^2}}} d\tau \right)^{-1}$$

The solution for x_2 involves another four factors with integrals of fractions with denominators having square roots of square roots. Instead of computing these solutions and certifying them in Isabelle, we perform differential induction. We show that I is an invariant by proving that the framed Fréchet derivatives on both sides of the inequality are 0. In Isabelle/HOL, certification of this reasoning is automatic due to our `dInduct_mega` tactic.

```
lemma "(x1^4*x2^2 + x1^2*x2^4 - 3*x1^2*x2^2 + 1 ≤ c) e ≤
  |{x1' = 2*x1^4*x2 + 4*x1^2*x2^3 - 6*x1^2*x2,
    x2' = -4*x1^3*x2^2 - 2*x1*x2^4 + 6*x1*x2^2}|
  (x1^4*x2^2 + x1^2*x2^4 - 3*x1^2*x2^2 + 1 ≤ c)"
  by dInduct_mega
```

Although the problem is simpler through differential induction, readers should be aware that the simplicity of proving this invariance in Isabelle benefits greatly from our increased automation. We describe below the automated steps to conclude the proof.

1. The inequality is transformed into a proof of invariance $\text{diff-inv } (x_1 \oplus x_2) f \top \mathbb{R}_+ 0 I$
2. Backward reasoning with (dinv-leq) requires showing $\mathcal{D}_{x_1 \oplus x_2}^f e \leq \mathcal{D}_{x_1 \oplus x_2}^f c$
3. The right hand side reduces to 0 while the simplifier performs the following rewrites on the left-hand side $e = x_1^4 \cdot x_2^2 + x_1^2 \cdot x_2^4 - 3 \cdot x_1^2 \cdot x_2^2 + 1$ (abbreviating $x'_i = \mathcal{D}_{x_1 \oplus x_2}^f x_i$)

$$\begin{aligned} \mathcal{D}_{x_1 \oplus x_2}^f e &= 4 \cdot x_1^3 \cdot x_2^2 \cdot x'_1 + 2 \cdot x_1^4 \cdot x_2 \cdot x'_2 + 2 \cdot x_1 \cdot x_2^4 \cdot x'_1 \\ &\quad + 4 \cdot x_1^2 \cdot x_2^3 \cdot x'_2 - 6 \cdot x_1 \cdot x_2^2 \cdot x'_1 - 6 \cdot x_1^2 \cdot x_2 \cdot x'_2 \\ &= 8 \cdot x_1^7 \cdot x_2^2 + 16 \cdot x_1^5 \cdot x_2^5 - 24 \cdot x_1^5 \cdot x_2^3 - 8 \cdot x_1^7 \cdot x_2^3 - 4 \cdot x_1^5 \cdot x_2^5 \\ &\quad + 12 \cdot x_1^5 \cdot x_2^3 + 4 \cdot x_1^5 \cdot x_2^5 + 8 \cdot x_1^3 \cdot x_2^7 - 12 \cdot x_1^3 \cdot x_2^5 - 16 \cdot x_1^5 \cdot x_2^5 \\ &\quad - 8 \cdot x_1^3 \cdot x_2^7 + 24 \cdot x_1^3 \cdot x_2^5 - 12 \cdot x_1^5 \cdot x_2^3 - 24 \cdot x_1^3 \cdot x_2^5 \\ &\quad + 36 \cdot x_1^3 \cdot x_2^3 + 24 \cdot x_1^5 \cdot x_2^3 + 12 \cdot x_1^3 \cdot x_2^5 - 36 \cdot x_1^3 \cdot x_2^3 \\ &= 0 \end{aligned}$$

4. Since $\mathcal{D}_{x_1 \oplus x_2}^f e = 0 \leq 0 = \mathcal{D}_{x_1 \oplus x_2}^f c$, the proof ends satisfactorily.

Thus, our tactic `dInduct_mega` hides various logical, algebraic, differential and numerical computations and certifications. This example showcases the scalability of hybrid systems verifications in interactive theorem provers. Namely, with adequate tactic implementations, ITPs become more automated tools and easier to use over time.

8.3 Reachability of a Rocket Launch

Consider a rocket's vertical liftoff and assume it loses fuel at a constant rate of $k > 0$ kilograms per second starting with $m_0 > k$ kilograms while its acceleration is equal to the amount of fuel left in it. These assumptions do not accurately model rockets' liftoff; however, they suffice to produce a behaviour approximating the observed phenomena, see Fig. 2, and facilitate the presentation of our contributions. The corresponding system of ODEs and its solution are

$$f = \begin{cases} m' = -k, \\ v' = m, \\ y' = v, \\ t' = 1, \end{cases} \quad \varphi \tau = \begin{cases} m \tau = -k \cdot \tau + m_0, \\ v \tau = -k \cdot \frac{\tau^2}{2} + m_0 \cdot \tau = \tau \cdot (-k \cdot \frac{\tau}{2} + m_0), \\ y \tau = -k \cdot \frac{\tau^3}{6} + m_0 \cdot \frac{\tau^2}{2} = \frac{\tau^2}{2} \cdot (-k \cdot \frac{\tau}{3} + m_0), \\ t \tau = \tau, \end{cases}$$

where m is the fuel's mass, v is the rocket's velocity, y is its altitude, and t models time.

We study the rocket's behaviour before it reaches a maximum altitude with its first-stage propulsion because the second stage should begin before the rocket starts falling. We therefore prove two things about the initial propulsion stage. The first is that no matter which height h we consider strictly below the maximum altitude $H = 2m_0^3/(3k^2)$, there will always be a state of the rocket greater than h approaching H . The second is that all scenarios with a single propulsion from the ground lead to altitudes lower than H . Using the abbreviations "odes" for f and "flow" for φ , the verification of the first specification is now possible thanks to our implementation of forward diamonds.

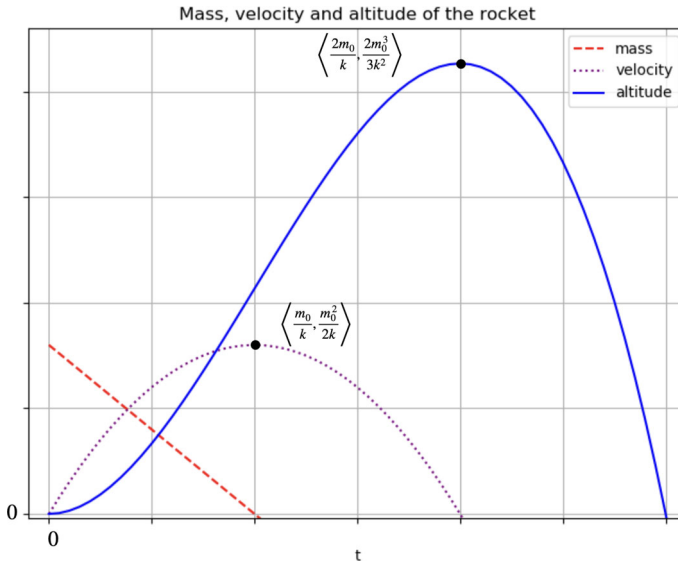


Fig. 2 Depiction of the rocket behaviour assuming $m_0 > k$

The proof starts with the law (fdia-flow) using the lemma `local_flow_on_rocket` asserting that φ is the flow for f . The proof of this is now automatic due to our tactic `local_flow_on_auto`. The second line of the proof for our specification culminates with some arithmetical reasoning, where we provide the time $2m_0/k$ as the witness for the existential quantifier in the law (fdia-flow). That is, the time for the second 0-intercept of the velocity when the maximum altitude is reached.

```
lemma local_flow_on_rocket:
  "local_flow_on [y↦$v, v↦$m, t↦1, m↦-k] (y+L v+L t+L m) UNIV UNIV flow"
  by local_flow_on_auto

lemma "(0 ≤ h ∧ h < H ∧ m = m_0 ∧ m_0 > k ∧ t = 0 ∧ v = 0 ∧ y = 0) ⇒
  using k_ge_1
  by (subst fdia_gode_frame_flow[OF local_flow_on_rocket]; expr_simp)
  (auto simp: field_simps power3_eq_cube intro!: exI[of _ "2*m_0/k"])"
```

The second specification looks equally simple and its three line proof is deceiving (see below). Our contributed tactics automatically handle VCG, derivative certifications, and uniqueness. However, as reported in Sect. 7, emerging arithmetic proof obligations often have to be checked separately. We do this with the lemma `rocket_arith` proved using our derivative tests from Sect. 6.7.

```
lemma "(m = m_0 ∧ m_0 > k ∧ t = 0 ∧ v = 0 ∧ y = 0) ⇒
  ≤ |ode| (y ≤ 2*m_0^3/(3*k^2))"
  apply (wlp_solve "flow")
  using k_ge_1 rocket_arith
  by (expr_simp add: le_fun_def)

lemma rocket_arith:
  assumes "(k::real) > 1" and "m_0 > k" and "x ∈ {0..}"
  shows "- k*x^3/6 + m_0*x^2/2 ≤ 2*m_0^3/(3*k^2)" (is "?lhs ≤ _")
proof-
```

```

let ?f = "λt. -k*t3/6 + m0*t2/2"
and ?f' = "λt. -k*t2/2 + m0*t"
and ?f'' = "λt. -k*t + m0"
have "2*m03/(3*k2) = -k*(2*m0/k)3/6 + m0*(2*m0/k)2/2" ( is "_ = ?rhs")
by (auto simp: field_simps power3_eq_cube)
moreover have "?lhs ≤ ?rhs"
proof(cases "x ≤ 2 * m0 / k")
case True
have ge_0_left: "0 ≤ y ⇒ y ≤ m0/k ⇒ ?f' 0 ≤ ?f' y" for y
apply (rule has_vderiv_mono_test(1)[of "{0..m0/k}" ?f' ?f'' 0])
using ⟨k > 1⟩ ⟨m0 > k⟩
by (auto intro!: vderiv_intros simp: field_simps)
moreover have ge_0_right: "m0/k ≤ y ⇒ y ≤ 2*m0/k
⇒ ?f' (2*m0/k) ≤ ?f' y" for y
apply (rule has_vderiv_mono_test(2)
[of "{m0/k..2*m0/k}" ?f' ?f'' - "2*m0/k"])
using ⟨k > 1⟩ ⟨m0 > k⟩
by (auto intro!: vderiv_intros simp: field_simps)
ultimately have ge_0: "∀y∈{0..2*m0/k}. 0 ≤ ?f' y"
using ⟨k > 1⟩ ⟨m0 > k⟩
by (fastforce simp: field_simps)
show ?thesis
apply (rule has_vderiv_mono_test(1)[of "{0..2*m0/k}" ?f ?f' - "2*m0/k"])
using ge_0 True ⟨x ∈ {0..}⟩ ⟨k > 1⟩ ⟨m0 > k⟩
by (auto intro!: vderiv_intros simp: field_simps)
next
case False
have "2*m0/k ≤ y ⇒ ?f' y ≤ ?f' (2*m0/k)" for y
apply (rule has_vderiv_mono_test(2)[of "{m0/k..}" ?f' ?f''])
using ⟨k > 1⟩ ⟨m0 > k⟩ by (auto intro!: vderiv_intros simp: field_simps)
hence obs: "∀y∈{2*m0/k..}. ?f' y ≤ 0"
using ⟨k > 1⟩ ⟨m0 > k⟩
by (clarsimp simp: field_simps)
show ?thesis
apply (rule has_vderiv_mono_test(2)[of "{2*m0/k..}" ?f ?f'])
using False ⟨k > 1⟩ obs
by (auto intro!: vderiv_intros simp: field_simps)
qed
ultimately show ?thesis
by simp
qed

```

Our proof of `rocket_arith` analyses the altitude behaviour to the left ($x \leq 2m_0/k$) and right ($x > 2m_0/k$) of the maximum altitude. Using the first derivative test, we show that, to the left, the altitude is increasing, and to the right, it is decreasing. Thus, the rocket achieves a maximum altitude at $(2m_0/k, 2m_0^3/(3k^2))$. Accordingly, we repeat our use of the derivative test to show that the velocity remains above 0 in the interval $[0, 2m_0/k]$.

This example illustrates the relevance of our additions of forward diamonds and derivative tests. The diamonds allow us to do reachability proofs and show progress for hybrid systems. The tests enable us to prove emerging real-arithmetic proof obligations after VCG and provide the basis for increasing automation in this subtask of the verification process.

We have showcased various examples illustrating the automation added to our hybrid system verification framework. We refer readers interested in more examples using, for instance, our integration of vectors and matrices to previous publications and our participation in ARCH competition reports [21, 34, 45, 47].



9 Related Work

We know of two complementary approaches to the verification of hybrid systems: reachability analysis and deductive verification. Reachability analysis [3, 5, 11, 14, 16, 26, 27, 41] approximates the set of all reachable states of a hybrid system via the iteration of its transition relation until reaching a fixed point or a specification-violating state. This approach iteratively explores a hybrid system's state space and finds states that violate specified properties.

Our focus is on the deductive verification of hybrid systems with interactive theorem provers (ITPs) [1, 6, 21, 28, 59, 64, 68, 73]. It uses mathematical proofs to establish adherence to safety specifications for all system states. Examples of this approach in relevant general-purpose ITPs include a formalisation of hybrid automata and invariant reasoning for them in the PVS prover [1], a shallowly embedded implementation of a Logic of Events for hybrid systems reasoning in the Coq prover [6], or the use of the Coquelicot library for formalising a temporal logic of actions with the same purpose [64]. All these approaches have different semantics from our predicate transformer one and arguably employ less automated ITPs than our choice. In Isabelle/HOL, Hoare-style verification and refinement frameworks have appeared [7, 40, 67] and fewer have been specialised to hybrid systems [20]. These frameworks could be combined with our own in the spirit of working towards a full hybrid systems development environment within Isabelle/HOL.

The formalisms to describe and prove hybrid systems' correctness specifications are as diverse as the tools to implement such deductive verification systems. The HHL and HHLPy provers [68, 73] employ a Hybrid Hoare Logic (HHL) [78] for reasoning about Hybrid Communicating Sequential Processes (HCSPs) with the duration calculus in Isabelle/HOL and Python respectively. Analogously, the KeYmaera and KeYmaera X provers implement versions of differential dynamic logic $d\mathcal{L}$, a logic to reason about hybrid systems [28, 59]. Our work has been compared with these provers in hybrid system verification competitions [45, 47] (see Sect. 7). Yet, the tools are very different in nature and implementation. While both families of provers implement specific logics, our development is flexible and includes a differential Hoare logic, a refinement calculus [20], rules from differential dynamic logic [21], and linear systems (matrix) integrations [34]. Moreover, both the HHL and KeYmaera families have integrated unverified tools into their certification process. The HHLPy prover is mainly written in Python, while KeYmaera X uses the Wolfram Engine and/or Z3 as black-box solvers for quantifying elimination procedures. Albeit, the first HHL prover is verified with Isabelle/HOL, and there is work towards verifying real arithmetic algorithms for integration into KeYmaera X [65]. In contrast, our work has taken a stricter approach where every input from external tools must be certified in Isabelle. This illustrates our long-term goal of enabling general-purpose ITPs with fully automated verification capabilities.

We built our IsaVODEs framework on top of the Archive of Formal Proofs (AFP) entry for Ordinary Differential Equations [37] and our own extensions to it [34, 50]. Together with Isabelle's HOL-Analysis library, they provide a thorough basis for stating real analysis theorems in Isabelle/HOL. In particular, the library already has a different formalisation of the fact that C^1 -differentiation implies Lipschitz continuity. Nevertheless, that version depends on a type of bounded linear continuous functions while our implementation avoids creating a new type and the corresponding abstraction functions. As a result, our version is more manageable within IsaVODEs.

Formalisations of $d\mathcal{L}$ and related logics have recently appeared in the AFP [12, 56] but they are not intended as verification tools and, therefore, they are incomparable with our framework. Despite the implementation differences, we compare KeYmaera X and our sup-

port for $d\mathcal{L}$ reasoning. Our personal experience indicates that using (one-step) differential cuts, weakenings, and inductions is similar to their use in KeYmaera X. However, supplying solutions works differently because our framework requires certifying or assuming their uniqueness. In contrast, the uniqueness of solutions is guaranteed by $d\mathcal{L}$'s syntax. Our work in this paper automates this certification process. Finally, our verification of the soundness of the differential ghost rule presented here could be generalised further (see Sect. 5.4) to match all the cases prescribed by $d\mathcal{L}$'s syntactic implementation.

10 Conclusions and Future Work

In this paper, we have described IsaVODEs, our framework for verifying cyber-physical systems in Isabelle/HOL. This substantial development includes both strong theoretical foundations and practical verification support, provided by automated theorem provers and an integration with computer algebra systems. Our language and verification technique extends $d\mathcal{L}$'s hybrid programs in several ways, notably with matrices to support engineering mathematics, and frames to support modular reasoning. We have validated our tool with a substantial library of benchmarks and examples.

Here we have improved our framework by formalising and proving VCG laws about forward diamonds which enable reasoning about the reachability or progress of hybrid systems. We have generalised our frame laws and $d\mathcal{L}$ -style differential ghosts rule, allowing us to derive related Darboux rules. We have formalised foundational theorems like the fact that differentiable functions are Lipschitz-continuous, and the first and second derivative test laws. These support the practical goal of increasing automation via our proof methods for performing differential induction or VCG via supplying flows. Our integration of CASs into this process makes the verification experience with flows seamless. Finally, we have evaluated the benefits of all these additions with various verification examples.

Our Isabelle-based approach is inherently extensible. We can add syntax and semantics for bespoke program operators and associated Hoare-logic rules to support tailoring for particular models. Overall, verifying CPSs using IsaVODEs benefits from the wealth of technology provided by Isabelle, notably the frontend, asynchronous document processing, the theory library, proof automation, and support for code generation. We need not be limited to a single notation but can provide semantics for established engineering notations. IsaVODEs benefits from the fact that Isabelle is a gateway for a variety of other verification tools through “hammers”, such as SMT solvers, model generators, and computer algebra systems. Our additions in this paper increase IsaVODEs' usability for complex verifications. We believe these advantages can allow the integration of our technology into software engineering workflows.

A limitation of our current approach occurs when the arithmetic obligations at the end of the verification are too complex for SMT solvers [53]. Currently, there are two options: users can manually prove these obligations themselves, or they can assert them at the cost of increasing uncertainty in their verification. In these cases, the ideal approach would connect tools deciding these expressions, e.g. CAS systems or domain-specific automated provers [4] in a way that IsaVODEs certifies the underlying reasoning. We leave this development for future work.

Another avenue of improvement, following our introduction of the forward diamond in our framework, is the addition of the remaining modal operators and their VCG rules [70]. Currently, we have only formalised a backward diamond but its VCG rules remain to be

proved. Their implementation could lead to a framework for incorrectness analysis [51] of hybrid systems complementing current testing and simulation techniques. We will also consider supporting further extensions to hybrid programs that provide additional modelling capacity such as (constructive) differential game logic and its Kaisar proof language [13, 60], quantified $d\mathcal{L}$, or differential-algebraic logic (DAL).

In terms of alternative uses of our framework, we expect $d\mathcal{L}$ -style security analysis about hybrid systems [77] to be easily done in IsaVODEs too. Similarly, IsaVODEs foundations have been used as semantics for other model-based robot development technologies [9, 15]. Therefore, IsaVODEs proofs could be integrated into these tools for increased confidence in the performed analysis.

A Appendix: ARCH2022 Evaluation Summary

The tables below summarise characteristics of the competition's verification problems [47] used for our evaluation in Sect. 7. In the final table, we provide basic statistics to describe the set of problems in its entirety: average number of proofs per problem, number of hybrid programs, regular programs or continuous problems, number of times the wlp strategy is used as opposed to the Hoare-style proof strategy, number of problems that required extra arithmetical facts, and usage of ODEs' solutions in comparison to other $d\mathcal{L}$ -methods.

ARCH22 BENCHMARKS EVALUATION												
Number	Name	Number of Proofs	Statement LOC	Proof LOC (shortest)	Proof LOC (longest)	Class	Proof Strategy	ODE strategy	Continuous Variables	Vars + Consts	ODE complexity	Proof used real arith helper lemma
1	Basic Assignment	1	1	1	1	1 Regular	wlp	-	0	1 None	no	-
2	Overwrite assignment on some branches	1	1	1	1	1 Regular	wlp	-	0	2 None	no	-
3	Overwrite assignment in loop	1	1	2	2	2 Regular	wlp	-	0	1 None	no	-
4	Overwrite assignment in ODE	3	1	1	3	3 Hybrid	wlp or hoare	solution or diff. induction	1	1 Linear	no	Both of our proof methods for differential induction used.
5	Overwrite with nondeterministic assignment	1	1	1	1	1 Regular	wlp	-	0	1 None	no	-
6	Tests and universal quantification	1	1	1	1	1 Regular	wlp	-	0	2 None	no	-
7	Overwrite assignment several times	2	3	7	15	15 Hybrid	hybrid	solution or diff. induction	1	2 Linear	no	-
8	Potentially overwrite dynamics	3	1	5	16	16 Hybrid	hybrid	solution or diff. induction	1	2 Linear	no	-
9	Potentially overwrite exponential decay	2	1	5	8	8 Hybrid	hybrid	solution or diff. ghosts	1	2 Linear	yes	-
10	Dynamics: Cascaded	2	1	5	11	11 Hybrid	wlp or hoare	solution or darboux	1	1 Linear	no	-
11	Dynamics: Single integrator time	2	1	1	1	1 Continuous	wlp or hoare	solution or diff. induction	1	1 Linear	no	-
12	Dynamics: Single integrator	2	1	1	6	6 Continuous	wlp or hoare	solution or diff. induction	2	2 Linear	no	-
13	Dynamics: Double integrator	1	1	1	1	1 Continuous	wlp	solution	3	3 Linear	no	-
14	Dynamics: Triple integrator	1	1	4	4	4 Continuous	hoare	diff. induction	4	4 Nonlinear	yes	-
15	Dynamics: Exponential decay (1)	2	1	1	3	3 Continuous	wlp or hoare	solution or diff. ghosts	1	1 Linear	no	-
16	Dynamics: Exponential decay (2)	2	1	4	8	8 Continuous	wlp or hoare	solution or darboux	1	1 Linear	no	-
17	Dynamics: Exponential decay (3)	1	1	1	1	1 Continuous	wlp	solution	1	2 Nonlinear	yes	-
18	Dynamics: Exponential growth (1)	2	1	1	11	11 Continuous	wlp or hoare	solution or diff. ghosts	1	1 Linear	no	-
19	Dynamics: Exponential growth (2)	1	1	2	2	2 Continuous	hoare	diff. induction	2	2 Nonlinear	no	-
20	Dynamics: Exponential growth (4)	1	1	1	1	1 Continuous	hoare	diff. induction	1	1 Nonlinear	no	-
21	Dynamics: Exponential growth (5)	1	1	15	15	15 Continuous	hoare	diff. induction	1	1 Nonlinear	no	-
22	Dynamics: Rotational dynamics (1)	2	1	1	3	3 Continuous	wlp or hoare	solution or diff. induction	2	2 Linear	no	-
23	Dynamics: Rotational dynamics (2)	1	1	11	11	11 Continuous	wlp or hoare	solution or diff. induction	3	3 Linear	yes	-
24	Dynamics: Rotational dynamics (3)	2	3	1	10	10 Continuous	wlp or hoare	solution or diff. induction	4	5 Linear	no	-
25	Dynamics: Spiral to equilibrium	1	1	11	11	11 Continuous	hoare	diff. induction	2	3 Linear	no	-
26	Dynamics: Open cases	1	1	16	16	16 Continuous	isabelle/hol	diff. induction	2	2 Nonlinear	no	-
27	Dynamics: Closed cases	1	1	8	8	8 Continuous	hoare	diff. induction	2	3 Nonlinear	no	-

ARCH22 BENCHMARKS EVALUATION													
Number	Name	Number of Proofs	Statement LOC	Proof LOC (shortest)	Proof LOC (longest)	Class	Proof Strategy	ODE strategy	Continuous Variables	Vars + Consts	ODE complexity	Proof used real arith helper lemma	Observation
28	Dynamics: Conserved quantity	1	3	4	4	Continuous	hoare	diff. induction	2	3	Nonlinear	no	-
29	Dynamics: Darboux equality	1	1	27	27	Continuous	isabelle/hoi	isabelle/hoi	2	4	Nonlinear	yes	-
30	Dynamics: Fractional Darboux equality	0	1	30	30	Continuous	hoare or isabelle/hoi	-	2	5	Nonlinear	no	Proof requires generalisation of Darboux rule
31	Dynamics: Darboux inequality	1	1	11	11	Continuous	wlp	solution	2	3	Nonlinear	yes	Proof of alternative statement
32	Dynamics: Bifurcation	1	1	24	24	Continuous	isabelle/hoi	isabelle/hoi	1	2	Nonlinear	no	-
33	Dynamics: Parametric switching between two different damped oscillators	1	8	23	23	Hybrid	wlp and hoare	diff. induction	2	5	Linear	yes	Helped by CAS for real arith
34	Dynamics: Nonlinear 1	1	1	1	1	Continuous	hoare	diff. induction	1	3	Nonlinear	no	-
35	Dynamics: Nonlinear 2	1	1	1	1	Continuous	hoare	diff. induction	2	3	Nonlinear	no	-
36	Dynamics: Nonlinear 4	1	1	3	3	Continuous	hoare	diff. induction	2	3	Nonlinear	no	-
37	Dynamics: Nonlinear 5	1	1	3	3	Continuous	hoare	diff. induction	2	3	Nonlinear	no	-
38	Dynamics: Riccati	1	1	1	1	Continuous	hoare	diff. induction	1	1	Nonlinear	no	-
39	Dynamics: Nonlinear differential cut	1	1	3	3	Continuous	hoare	diff. induction	2	2	Nonlinear	no	-
40	STTT Tutorial: Example 1	2	1	1	7	Continuous	wlp or hoare	solution or diff. induction	2	3	Linear	no	-
41	STTT Tutorial: Example 2	1	6	2	2	Hybrid	wlp	solution	2	5	Linear	no	-
42	STTT Tutorial: Example 3a	1	10	5	5	Hybrid	wlp	solution	2	6	Linear	yes	Did not count real arith. lemmata
43	STTT Tutorial: Example 4a	1	10	1	1	Hybrid	wlp	solution	2	5	Linear	no	-
44	STTT Tutorial: Example 4b	1	6	1	1	Hybrid	wlp	solution	2	5	Linear	no	-
45	STTT Tutorial: Example 4c	1	11	1	1	Hybrid	wlp	solution	2	5	Linear	no	Did not count real arith. lemmata
46	STTT Tutorial: Example 5	1	11	3	3	Hybrid	wlp	solution	3	8	Linear	yes	Did not count real arith. lemmata
47	STTT Tutorial: Example 6	1	11	8	8	Hybrid	wlp	solution	3	8	Linear	yes	Did not count real arith. lemmata
48	STTT Tutorial: Example 7	1	11	8	8	Hybrid	wlp	solution	3	8	Linear	yes	Did not count real arith. lemmata
49	STTT Tutorial: Example 9a	1	3	16	16	Hybrid	hoare	diff. induction	2	6	Linear	yes	Did not count real arith. lemmata
50	STTT Tutorial: Example 9b	1	10	46	46	Hybrid	hoare	diff. induction	2	7	Linear	yes	Helped by CAS for real arith
51	STTT Tutorial: Example 10	1	21	98	98	Hybrid	hoare	diff. induction	6	13	Nonlinear	yes	Did not count real arith. lemmata
52	LCS: Example 1 Continuous car accelerates forward	1	1	2	2	Continuous	wlp	solution	2	3	Linear	no	-

ARCH22 BENCHMARKS EVALUATION													
Number	Name	Number of Proofs	Statement LOC	Proof LOC (shortest)	Proof LOC (longest)	Class	Proof Strategy	ODE strategy	Continuous Variables	Vars + Consts	ODE complexity	Proof used real arith helper lemma	Observation
53	LCS: Example 2 Single car drives forward	1	8	1	1	Hybrid	wlp	solution	2	5	Linear	no	-
54	LCS: Example 3a event-triggered car drives forward	1	9	1	1	Hybrid	wlp	solution	2	5	Linear	no	-
55	LCS: Example 4a safe stopping of time-triggered car	1	7	10	10	Hybrid	wlp	solution	3	8	Linear	yes	Did not count real arith. lemmata
56	LCS: Example 4b progress of time-triggered car	0	7	-	-	Hybrid	wp and isabelle/hoi	-	3	7	Linear	-	Do not know the proof
57	LCS: Example 4c relative safety of time-triggered car	1	3	9	9	Hybrid	wlp	solution	3	8	Linear	yes	Did not count real arith. lemmata
58	LCS: Example 5	1	1	2	2	Hybrid	wlp	solution	2	4	Linear	yes	Did not count real arith. lemmata
59	Controlability Equivalence	1	4	10	10	Hybrid	wlp	solution	3	7	Linear	yes	Helped by CAS for real arith
60	Acceleration Equivalence	1	4	10	10	Hybrid	wlp	solution	3	7	Linear	yes	Helped by CAS for real arith
61	Model Predictive Control Design Car	1	12	10	10	Hybrid	wlp	solution	3	7	Linear	yes	Did not count real arith. lemmata
62	ETCS: Essentials (safety)	1	9	6	6	Hybrid	wlp	solution	3	8	Linear	yes	Did not count real arith. lemmata
63	ETCS: Proposition 1 (Controlability)	1	5	3	3	Hybrid	wlp	solution	2	5	Linear	yes	Helped by CAS for real arith
64	ETCS: Proposition 4 (Reactivity)	0	4	-	-	Hybrid	wlp	solution	3	8	Linear	-	Do not know the proof
65	Harmonic oscillator	1	7	6	6	Hybrid	matrices and isabelle/hoi	solution	2	4	Linear	yes	Did not count real arith. lemmata
66	Benchmarks/Nonlinear/Ahm ad Parillo Krstic	1	3	21	21	Continuous	hoare	diff. ghosts	2	2	Nonlinear	yes	Did not count real arith. lemmata
67	Benchmarks/Nonlinear/Kroenrich Place Fig_9_11 page 83	0	3	-	-	Continuous	hoare	-	2	2	Nonlinear	-	Proof requires generalisation of Darboux
Average		1.1818182	3.7272727	8.1111111	9.19047619	Hybrid		29 Average	1.9242424	3.8030303			
Median		1	1	3	5	Continuous		32 Median	2	3			
Mode		1	1	1	1	Regular		5 Mode	2	2			
Sum		78	246	511	579	wlp		39 Sum	127	251			
Max		3	21	98	98	hoare		31 Max	6	13			
Min		0	1	1	1	solution		37 Min	0	1			
						induction		26					
						ghosts		4					
						darboux		2					
						isabelle		3					
						extra real arith		23					
						>1 proofs		14					

Acknowledgements We thank the anonymous reviewers for their valuable feedback that helped our article convey its contributions more clearly. The first author would also like to thank Esaú Alejandro Pérez Rosales for his work developing tactics for the simplification of monomials in Isabelle that inspired some of those employed in this article.

Author Contributions Jonathan Julián Huerta y Munive developed and formalised in Isabelle/HOL the theory related to the verification of hybrid systems. He also wrote some of the automated tactics of the framework and evaluated the framework. Simon Foster formalised the theory related to the framework's modelling language and wrote other automated tactics. Mario Gleirscher and Georg Struth participated in the conference version of this article and revised the final version of the article. Mario provided the model and the simulation of the autonomous marine vehicle example. Georg Struth also developed the theory behind the framework's verification principles. Christian Pardillo Laursen and Thomas Hickman integrated Mathematica and SageMath respectively into the framework.

Funding A Novo Nordisk Fonden Start Package Grant (NNF20OC0063462) and a Horizon MSCA 2022 Postdoctoral Fellowship (Project acronym DeepIsaHOL and Number 101102608) partially supported the first author during the development of this article. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Executive Agency. Neither the European Union nor the European Research Executive Agency can be held responsible for them. The work was also funded by UKRI-EPSRC project CyPhyAssure (Grant reference EP/S001190/1), the Assuring Autonomy International Programme (AAIP; Grant CSI:Cobot), a partnership between Lloyd's Register Foundation and the University of York, and Labex DigiCosme through an invited professorship of the fourth author at the Laboratoire d'informatique de l'École Polytechnique.

Data Availability No datasets were generated or analysed during the current study.

Declarations

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

1. Ábrahám-Mumm, E., Steffen, M., Hannemann, U.: Verification of hybrid systems: formalization and proof rules in PVS. In: ICECCS 2001, pp. 48–57. IEEE Computer Society, New Jersey (2001)
2. Ackerman, E., Gatewood, L.C., Rosevear, J.W., Molnar, G.D.: Model studies of blood-glucose regulation. *Bull. Math. Biophys.* **27**(5), 21–37 (1965). <https://doi.org/10.1007/BF02477259>
3. Adimoolam, A.S., Dang, T.: Safety verification of networked control systems by complex zonotopes. *Leibniz Trans. Embed. Syst.* **8**(2), 01–10122 (2022). <https://doi.org/10.4230/LITES.8.2.1>
4. Akbarpour, B., Paulson, L.C.: MetiTarski: an automatic theorem prover for real-valued special functions. *J. Autom. Reason.* **44**(3), 175–205 (2010). <https://doi.org/10.1007/S10817-009-9149-2>
5. Althoff, M.: An introduction to CORA 2015. In: ARCH 2015, vol. 34, pp. 120–151. EasyChair, EasyChair (2015). <https://doi.org/10.29007/zbkv>
6. Anand, A., Knepper, R.A.: Roscoq: robots powered by constructive reals. In: ITP. LNCS, vol. 9236, pp. 34–50. Springer, Heidelberg (2015)
7. Armstrong, A., Gomes, V.B.F., Struth, G.: Building program construction and verification tools from algebraic principles. *Formal Aspects Comput.* **28**(2), 265–293 (2016)
8. Back, R., Wright, J.: *Refinement Calculus—A Systematic Introduction*. Springer, Heidelberg (1998)
9. Baxter, J., Carvalho, G., Cavalcanti, A., Júnior, F.R.: Roboworld: verification of robotic systems with environment in the loop. *Formal Aspects Comput.* (2023). <https://doi.org/10.1145/3625563>. (Just Accepted)
10. Blanchette, J.C., Kaliszyk, C., Paulson, L.C., Urban, J.: Hammering towards QED. *J. Formaliz. Reason.* **9**(1) (2016)

11. Bogomolov, S., Forets, M., Frehse, G., Potomkin, K., Schilling, C.: JuliaReach: a toolbox for set-based reachability. In: Ozay, N., Prabhakar, P. (eds.) HSCC 2019, pp. 39–44. ACM, New York (2019). <https://doi.org/10.1145/3302504.3311804>
12. Bohrer, R., Rahli, V., Vukotic, I., Völz, M., Platzer, A.: Formally verified differential dynamic logic. In: CPP, pp. 208–221. ACM, New York (2017)
13. Bohrer, R., Platzer, A.: Structured proofs for adversarial cyber-physical systems. ACM Trans. Embed. Comput. Syst. **20**(5s), 1–26 (2021). <https://doi.org/10.1145/3477024>
14. Bu, L., Li, Y., Wang, L., Chen, X., Li, X.: BACH 2 : bounded reachability checker for compositional linear hybrid systems. In: DATE 2010, pp. 1512–1517. IEEE Computer Society, New Jersey (2010)
15. Cavalcanti, A., Attala, Z., Baxter, J., Miyazawa, A., Ribeiro, P.: Model-based engineering for robotics with RoboChart and RoboTool, pp. 106–151. Springer, Heidelberg (2023). https://doi.org/10.1007/978-3-031-43678-9_4
16. Chen, X., Abraham, E., Sankaranarayanan, S.: Flow*: an analyzer for non-linear hybrid systems. In: CAV 2013. LNCS, vol. 8044, pp. 258–263. Springer, Heidelberg (2013)
17. Cordwell, K., Yong, K.T., A., P.: A verified decision procedure for univariate real arithmetic with the BKR algorithm. In: Cohen, L., Kaliszyk, C. (eds.) ITP. Leibniz International Proceedings in Informatics (LIPIcs), vol. 193, pp. 14–11420. Schloss Dagstuhl—Leibniz-Zentrum für Informatik, Germany (2021)
18. Foster, S., Baxter, J.: Automated algebraic reasoning for collections and local variables with lenses. In: RAMiCS. LNCS, vol. 12062. Springer, Heidelberg (2020)
19. Foster, S., Gleirscher, M., Calinescu, R.: Towards deductive verification of control algorithms for autonomous marine vehicles. In: ICECCS. IEEE, New Jersey (2020)
20. Foster, S., Huerta y Munive, J.J., Struth, G.: Differential Hoare logics and refinement calculi for hybrid systems with Isabelle/HOL. In: RAMiCS[postponed]. LNCS, vol. 12062, pp. 169–186 (2020). https://doi.org/10.1007/978-3-030-43520-2_11
21. Foster, S., Munive, J.J.H., Gleirscher, M., Struth, G.: Hybrid systems verification with Isabelle/HOL: simpler syntax, better models, faster proofs. In: FM 2021. LNCS, vol. 13047, pp. 367–386. Springer, Heidelberg (2021). https://doi.org/10.1007/978-3-030-90870-6_20
22. Foster, S., Nemouchi, Y., Gleirscher, M., Wei, R., Kelly, T.: Integration of formal proof into unified assurance cases with Isabelle/SACM. Formal Aspects Comput. (2021)
23. Foster, S., Zeyda, F.: Optics. Archive of Formal Proofs (2017)
24. Foster, J.: Bidirectional programming languages. PhD thesis, University of Pennsylvania (2009)
25. Foster, S., Baxter, J., Cavalcanti, A., Woodcock, J., Zeyda, F.: Unifying semantic foundations for automated verification tools in Isabelle/UTP. Sci. Comput. Programm. **197**, 102510 (2020). <https://doi.org/10.1016/j.scico.2020.102510>
26. Frehse, G., Guernic, C.L., Donzé, A., Cotton, S., Ray, R., Lebeltel, O., Ripado, R., Girard, A., Dang, T., Maler, O.: SpaceEx: Scalable verification of hybrid systems. In: CAV 2011. LNCS, vol. 6806, pp. 379–395. Springer, Heidelberg (2011)
27. Frehse, G., Krogh, B.H., Rutenbar, R.A., Maler, O.: Time domain verification of oscillator circuit properties. In: Maler, O. (ed.) FAC 2005. ENTCS, vol. 153, pp. 9–22. Elsevier, Amsterdam (2005). <https://doi.org/10.1016/j.entcs.2006.02.019>
28. Fulton, N., Mitsch, S., Quesel, J.-D., Völz, M., Platzer, A.: KeYmaera X: an axiomatic tactical theorem prover for hybrid systems. In: Felty, A.P., Middeldorp, A. (eds.) CADE. LNCS, vol. 9195, pp. 527–538. Springer, Heidelberg (2015). https://doi.org/10.1007/978-3-319-21401-6_36
29. Gallicchio, J., Tan, Y.K., Mitsch, S., Platzer, A.: Implicit definitions with differential equations for keymaera X—(system description). In: IJCAR 2022, pp. 723–733 (2022). https://doi.org/10.1007/978-3-031-10769-6_42
30. Gomes, V.B.F., Struth, G.: Modal Kleene algebra applied to program correctness. In: FM. LNCS, vol. 9995, pp. 310–325 (2016)
31. Harel, D., Kozen, D., Tiuryn, J.: Dynamic Logic. MIT Press, Cambridge (2000)
32. Hölzl, J., Immler, F., Huffman, B.: Type classes and filters for mathematical analysis in Isabelle/HOL. In: ITP. LNCS, vol. 7998, pp. 279–294. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-39634-2_21
33. Hölzl, J.: Proving inequalities over reals with computation in Isabelle/HOL. In: PLMMS, pp. 38–45. ACM, New York (2009)
34. Huerta y Munive, J.J.: Affine systems of ODEs in Isabelle/HOL for hybrid-program verification. In: SEFM. LNCS, vol. 12310, pp. 77–92. Springer, Heidelberg (2020). https://doi.org/10.1007/978-3-030-58768-0_5
35. Immler, F., Hölzl, J.: Ordinary differential equations. Archive of Formal Proofs (2012)

36. Immler, F., Traut, C.: The flow of ODEs: Formalization of variational equation and poincaré. In: ITP 2016. LNCS, vol. 9807, pp. 184–199. Springer, Heidelberg (2016). <https://doi.org/10.1007/s10817-018-9449-5>
37. Immler, F., Traut, C.: The flow of ODEs: formalization of variational equation and Poincaré map. *J. Autom. Reason.* **62**(2), 215–236 (2019)
38. Jeannin, J., Ghorbal, K., Kouskoulas, Y., Schmidt, A., Gardner, R., Mitsch, S., Platzer, A.: A formally verified hybrid system for safe advisories in the next-generation airborne collision avoidance system. *STTT* **19**(6), 717–741 (2017). <https://doi.org/10.1007/s10009-016-0434-1>
39. Kuncar, O., Popescu, A.: A consistent foundation for Isabelle/HOL. *J. Autom. Reason.* **62**, 531–555 (2019)
40. Lammich, P.: Refinement to imperative HOL. *J. Autom. Reason.* **62**(4), 481–503 (2019). <https://doi.org/10.1007/s10817-017-9437-1>
41. Li, Y., Zhu, H., Braught, K., Shen, K., Mitra, S.: Verse: A python library for reasoning about multi-agent hybrid system scenarios. In: CAV 2023. LNCS, vol. 13964, pp. 351–364. Springer, Heidelberg (2023). https://doi.org/10.1007/978-3-031-37706-8_18
42. Li, W., Passmore, G., Paulson, L.: Deciding univariate polynomial problems using untrusted certificates in Isabelle/HOL. *J. Autom. Reason.* **62**, 29–91 (2019)
43. Loos, S.M., Platzer, A., Nistor, L.: Adaptive cruise control: hybrid, distributed, and now formally verified. In: FM 2011. LNCS, vol. 6664, pp. 42–56. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-21437-0_6
44. Matichuk, D., Murray, T.C., Wenzel, M.: Eisbach: a proof method language for Isabelle. *J. Autom. Reason.* **56**(3), 261–282 (2016)
45. Mitsch, S., Huerta y Munive, J.J., Jin, X., Zhan, B., Wang, S., Zhan, N.: ARCH-COMP20 category report: hybrid systems theorem proving. In: ARCH20., vol. 74, pp. 141–161. EasyChair, EasyChair (2020). <https://doi.org/10.29007/bdq9>
46. Mitsch, S., Platzer, A.: Verified runtime validation for partially observable hybrid systems. (2018). [arXiv:1811.06502](https://arxiv.org/abs/1811.06502)
47. Mitsch, S., Zhan, B., Sheng, H., Bentkamp, A., Jin, X., Wang, S., Foster, S., Laursen, C.P., Munive, J.J.H.: ARCH-COMP22 category report: hybrid systems theorem proving. In: ARCH22. EPiC Series in Computing, vol. 90, pp. 185–203. EasyChair, Munich (2022). <https://doi.org/10.29007/4lxf>
48. Miyazawa, A., Ribeiro, P., Li, W., Cavalcanti, A., Timmis, J., Woodcock, J.: RoboChart: modelling and verification of the functional behaviour of robotic applications. *Softw. Syst. Model.* **18**, 3097–3149 (2019)
49. Munive, J.J.: Algebraic verification of hybrid systems in Isabelle/HOL. PhD thesis, The University of Sheffield (2021)
50. Munive, J.J., Struth, G.: Predicate transformer semantics for hybrid systems. *J. Autom. Reason.* **66**(1), 93–139 (2022). <https://doi.org/10.1007/s10817-021-09607-x>
51. O’Hearn, P.W.: Incorrectness logic. In: Proc. ACM Program. Lang. 4(POPL), 10–11032 (2020). <https://doi.org/10.1145/3371078>
52. Oles, F.: A category-theoretic approach to the semantics of programming languages. PhD thesis, Syracuse University (1982)
53. Paulson, L.C., Blanchette, J.C.: Three years of experience with Sledgehammer, a practical link between automatic and interactive theorem provers. In: Sutcliffe, G., Schulz, S., Ternovska, E. (eds.) IWIL 2010. EPiC Series in Computing, vol. 2, pp. 1–11. EasyChair, EasyChair (2010). <https://doi.org/10.29007/36dt>
54. Platzer, A., Quesel, J.: European train control system: a case study in formal verification. In: ICFEM. LNCS, vol. 5885, pp. 246–265. Springer, Heidelberg (2009)
55. Platzer, A., Tan, Y.K.: Differential equation axiomatization: the impressive power of differential ghosts. In: LICS, pp. 819–828. ACM, New Jersey (2018)
56. Platzer, A.: Differential game logic. *Archive of Formal Proofs* (2019)
57. Platzer, A.: Logics of dynamical systems. In: LICS, pp. 13–24 (2012). <https://doi.org/10.1109/LICS.2012.13>
58. Platzer, A.: The structure of differential invariants and differential cut elimination. *Logical Methods Comput. Sci.* **8**(4) (2008)
59. Platzer, A.: *Logical Analysis of Hybrid Systems*. Springer, Heidelberg (2010)
60. Platzer, A.: Differential game logic. *ACM TOCL* **17**(1), 1–1152 (2015)
61. Platzer, A.: *Logical Foundations of Cyber-Physical Systems*. Springer, Heidelberg (2018)
62. Quesel, J., Mitsch, S., Loos, S.M., Arechiga, N., Platzer, A.: How to model and prove hybrid systems with KeYmaera: a tutorial on safety. *Int. J. Softw. Tools Technol. Transf.* **18**(1), 67–91 (2016)
63. Reynolds, J.: Separation logic: a logic for shared mutable data structures. In: LICS. IEEE, New Jersey (2002)

64. Ricketts, D., Malecha, G., Alvarez, M.M., Gowda, V., Lerner, S.: Towards verification of hybrid systems in a foundational proof assistant. In: MEMOCODE, pp. 248–257. IEEE, New Jersey (2015)
65. Scharager, M., Cordwell, K., Mitsch, S., Platzer, A.: Verified quadratic virtual substitution for real arithmetic. In: Huisman, M., Pasareanu, C.S., Zhan, N. (eds.) FM 2021. LNCS, vol. 13047, pp. 200–217. Springer, Heidelberg (2021). https://doi.org/10.1007/978-3-030-90870-6_11
66. Schirmer, N., Wenzel, M.: State spaces—the locale way. In: SSV 2009. ENTCS, vol. 254, pp. 161–179 (2009)
67. Schirmer, N.: Verification of sequential imperative programs in isabelle-hol. PhD thesis, Technical University Munich, Germany (2006)
68. Sheng, H., Bentkamp, A., Zhan, B.: HHLPy: practical verification of hybrid systems using hoare logic. In: Chechik, M., Katoen, J., Leucker, M. (eds.) FM 2023. LNCS, vol. 14000, pp. 160–178. Springer, Heidelberg (2023). https://doi.org/10.1007/978-3-031-27481-7_11
69. Stanescu, D., Chen-Charpentier, B.M.: Random coefficient differential equation models for bacterial growth. *Math. Comput. Model.* **50**(5), 885–895 (2009). <https://doi.org/10.1016/j.mcm.2009.05.017>
70. Struth, G.: Transformer semantics. *Archive of Formal Proofs* (2018)
71. Teschl, G.: Ordinary Differential Equations and Dynamical Systems. AMS, Rhode Island (2012)
72. The Sage Developers: SageMath, the Sage Mathematics Software System (Version 9.0). (2020)
73. Wang, S., Zhan, N., Zou, L.: An improved HHL prover: an interactive theorem prover for hybrid systems. In: ICFEM. LNCS, vol. 9407, pp. 382–399 (2015)
74. Weinert, F.: Radioactive Decay Law (Rutherford–Soddy), pp. 630–632. Springer, Berlin (2009). https://doi.org/10.1007/978-3-540-70626-7_183
75. Wolfram Research: Expressions—Wolfram Language. <https://reference.wolfram.com/language/guide/Expressions.html>. Accessed 11 Jan 2024
76. Wolfram Research: Wolfram Engine. <https://www.wolfram.com/engine/>. Accessed 05 Jan 2024
77. Xiang, J., Fulton, N., Chong, S.: Relational analysis of sensor attacks on cyber-physical systems. In: CSF 2021, pp. 1–16. IEEE, New Jersey (2021). <https://doi.org/10.1109/CSF51468.2021.00035>
78. Zhan, N., Zhan, B., Wang, S., Guelev, D.P., Jin, X.: A generalized hybrid Hoare logic. *CoRR arXiv:2303.15020* (2023). <https://doi.org/10.48550/arXiv.2303.15020>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.